Ninsho: An Authentication and Session Management SDK for Node.js

Project Report

Submitted in fulfillment of the requirements

for the degree of

Bachelor of Technology

in

Information Technology

by

Jain Darshan Arvind (24101B2002)
Jadhav Yash Gangadhar (24101B2003)
Padhi Chetna Shivshankar (24101B2005)
Sharma Chirag Pradeep (24101C2007)

Project Guide

Dr. Varsha Bhosale



Department of Information Technology
Vidyalankar Institute of Technology
Wadala (E), Mumbai – 400037
(Affiliated to University of Mumbai)

2025-2026

Certificate

This is to certify that the project entitled

“Ninsho: An Authentication and Session Management SDK for Node.js”

is a bonafide work done by

Jain Darshan Arvind (24101B2002)
Jadhav Yash Gangadhar (24101B2003)
Padhi Chetna Shivshankar (24101B2005)
Sharma Chirag Pradeep (24101C2007)

and is submitted in fulfillment of the requirement for the degree of

Bachelor of Technology

in Information Technology

to the

Vidyalankar Institute of Technology, Wadala, Mumbai.

Project Guide	Head of Department	Principal

Dissertation Approval for B.Tech.

This is to certify that the dissertation entitled “Ninsho: An Authentication and Session Management SDK for Node.js” is a bonafide work done by [Student Names and Roll Numbers] under the supervision of [Dr. / Prof. Supervisor Name]. This dissertation has been approved for the award of the B.Tech. degree in Information Technology, Vidyalankar Institute of Technology, Wadala, Mumbai.

Examiners:

1. _____

2. _____

Date: _____

Place: _____

Declaration

I declare that this written submission represents my ideas in my own words and where others' ideas or words have been included, I have adequately cited and referenced the original sources. I also declare that I have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in my submission. I understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

1. _____

2. _____

3. _____

4. _____

(Roll No., Name and Signature of Students)

Date: _____

Place: _____

Acknowledgement

We express our sincere gratitude to our project guide, Dr. Varsha Bhosale, Department of Information Technology, for guidance, encouragement and critical feedback throughout this project.

We thank the Head of the Department, Dr. Vidya Chitre, the Principal, Dr. Sangeeta Joshi, and the faculty and laboratory staff of Vidyalankar Institute of Technology for the facilities and support provided. We also thank our families and friends for their constant support.

Jain Darshan Arvind (24101B2002)
Jadhav Yash Gangadhar (24101B2003)
Padhi Chetna Shivshankar (24101B2005)
Sharma Chirag Pradeep (24101C2007)

Abstract

Web applications often assemble authentication from loosely coupled token libraries, which leaves gaps in revocation, refresh-token theft handling and replay of stolen bearer credentials. This project presents Ninsho, an open-source authentication and session-management SDK for Node.js, and evaluates it against its own documented security properties. Ninsho issues opaque access tokens verified by a single server-side lookup, rotates single-use refresh tokens with reuse detection, optionally issues PASETO v4.public tokens, binds tokens to a browser-held key using DPoP proof-of-possession with replay protection, provides ownership and role guards, and includes an optional WebAuthn verifier. Session state is held in an injected store, with Redis as the production implementation. The evaluation combines more than two thousand automated tests, recorded as passing without skips in the project's own runs against a dedicated Redis 7 instance, a continuous-integration run in which all eight jobs, including Chromium, Firefox and WebKit regressions, succeeded, twenty-five recorded manual attack entries, Redis restart, outage and data-loss scenarios, and SecureShop, a demonstration application that contrasts a deliberately limited stateless JWT baseline with Ninsho. The results support the documented behaviours under the tested conditions; they do not prove security. Revoked PASETO tokens can become valid again if Redis state is lost. A fresh local re-run on 21 October 2026 passed 2,152 automated tests with real Redis, and a single-machine benchmark compared Ninsho with a JWT baseline that performs an equivalent Redis revocation check.

Keywords: Authentication, Session Management, Refresh-Token Rotation, DPoP, PASETO, WebAuthn, Redis

Contents

Abstract vi

List of Figures x

List of Tables xi

Abbreviations xiii

Chapter 1 Introduction 1
1.1 Background 1
1.1.1 Identity verification, authentication, authorization and session management 2
1.1.2 Access tokens, refresh tokens and their lifetimes 3
1.1.3 Practical problems in session lifecycle management 3
1.2 Motivation 4
1.3 Problem Statement 4
1.4 Objectives 5
1.5 Scope of the Work 5
1.6 Intended Users, Assumptions and Constraints 6
1.7 Requirements 7
1.7.1 Functional requirements 7
1.7.2 Non-functional requirements 7
1.8 Requirements Traceability 8
1.9 Organization of the Report 9

Chapter 2 Literature Survey 10
2.1 Review of Existing Work 10
2.1.1 Authentication schemes and web sessions 10
2.1.2 OAuth, token theft and refresh-token security 10
2.1.3 JWT security and implementation quality 11
2.1.4 Proof-of-possession and sender-constrained tokens 11
2.1.5 Passkeys and WebAuthn 11
2.1.6 Server-side sessions, opaque tokens and introspection 11
2.1.7 PASETO and the removal of algorithm agility 12
2.1.8 Revocation strategies and their consistency trade-offs 12
2.1.9 Session hijacking and consistency across tabs and devices 13
2.2 Summary of Literature 13
2.3 Research Gaps 14
2.4 Comparison of Token and Session-Management Designs 15
2.5 Positioning of Ninsho Relative to Prior Work 15

Chapter 3 Proposed Methodology 17
3.1 System Architecture 17
3.1.1 Token engines and the one-read verification path 18
3.1.2 Session manager, refresh tokens and the store 19
3.1.3 Authorization guards and framework integration 19

3.1.4 Browser client and cross-tab invalidation 20
3.1.5 WebAuthn verification 20
3.2 Test Data and Experimental Inputs 21
3.3 Algorithm Description 21
3.3.1 Session creation and request verification 21
3.3.2 Refresh-token rotation and reuse detection 22
3.3.3 Revocation 23
3.3.4 DPoP proof verification 23
3.3.5 Evaluation Metrics 24
3.4 Tools and Technologies Used 24
3.5 System Context and Deployment 25
3.6 Threat Model 27
3.6.1 Assets 27
3.6.2 Adversaries and trust boundaries 28
3.6.3 Security objectives and out-of-scope threats 29
3.7 Detailed Lifecycle Design 29
3.7.1 Session creation and access-token verification 29
3.7.2 Refresh-token rotation and reuse detection 31
3.7.3 Logout and revocation 32
3.7.4 DPoP proof validation and replay protection 33
3.7.5 Authorization guards 33
3.7.6 Rate limiting and single-use tokens 34
3.7.7 WebAuthn verification responsibilities 34
3.8 Implementation Details 35
3.8.1 Public interface of the server facade 35
3.8.2 Configuration and secure defaults 35
3.8.3 Store contract and key schema 36
3.8.4 Error model and audit events 37
3.8.5 Integration responsibilities 38
3.8.6 Security controls and their limits 39
3.9 SecureShop Consumer Showcase 39

Chapter 4 Results and Discussion 41
4.1 Experimental Setup 41
4.2 Results 42
4.2.1 Automated test suite and continuous integration 42
4.2.2 Security behaviours and their test evidence 43
4.2.3 Manual attack procedures 44
4.2.4 Browser evidence 45
4.2.5 Redis persistence, outage and data loss 45
4.2.6 SecureShop demonstration 46
4.2.7 Historical benchmark measurements 47
4.2.8 Organization of the automated test suite 48
4.2.9 Selected test cases 49
4.2.10 Regression tests added during the final review 50
4.2.11 Fresh benchmark: objectives, configurations and procedure 50

4.2.12 Fresh benchmark: verification cost 51
4.2.13 Fresh benchmark: HTTP throughput and variability 52
4.3 Comparison with Existing Methods 54
4.4 Limitations and Threats to Validity 55
4.5 Security Evaluation Against the Threat Model 56
4.5.1 Findings of the review 56
4.5.2 Implications for deployment 57
4.6 Critical Discussion 58

Chapter 5 Conclusion and Future Scope 60
5.1 Conclusion 60
5.2 Technical Contributions 61
5.3 Lessons Learned 62
5.4 Future Scope 62
5.5 Validation Required Before Production-Readiness Claims 63

Chapter 6 List of Publications 64
6.1 List of Published Papers 64
6.2 Details of Participation in Project Competitions 64

References 65

List of Figures

Figure 3.1: Block diagram of the Ninsho architecture....17
Figure 3.2: Opaque token verification: one store read per request....18
Figure 3.3: Cross-tab invalidation after a confirmed logout....20
Figure 3.4: Refresh-token states, grace window and reuse detection....22
Figure 3.5: DPoP proof validation pipeline....23
Figure 3.6: Evidence sources used in this report....25
Figure 3.7: System context of a deployment that uses Ninsho....25
Figure 3.8: Package architecture and runtime dependencies (release 0.1.1)....26
Figure 3.9: Multi-instance deployment with a shared Redis store and synchronized browser tabs....27
Figure 3.10: Trust boundaries and adversaries considered in the threat model....28
Figure 3.11: Sequence of session creation and verification of an opaque access token....30
Figure 3.12: Sequence of refresh-token rotation with a parallel tab and a later replay....31
Figure 3.13: Sequence of session revocation and the marker that guards a concurrent rotation....33
Figure 3.14: Division of responsibilities in a WebAuthn registration and sign-in....35
Figure 4.1: Tests per workspace in the recorded run (project-recorded)....43
Figure 4.2: SecureShop, Comparison A: replay of a copied JWT after client logout....46
Figure 4.3: SecureShop, Comparison C: token-only, wrong-key and replayed-proof requests rejected....47
Figure 4.4: Historical verification throughput, release 0.1.0 (not for the current commit)....48
Figure 4.5: Per-trial median verification latency (dots) and the median of the three trials (bars)....52
Figure 4.6: Per-trial throughput (dots, 5 s each) and medians (bars); starred configurations are references without an equivalent revocation guarantee....53
Figure 4.7: Throughput at 16 clients against the configuration's position in its trial's run order....54

List of Tables

Table 1.1: Session-management problems and the corresponding Ninsho mechanisms....3
Table 1.2: Assumptions and constraints 6
Table 1.3: Functional requirements 7
Table 1.4: Non-functional requirements 7
Table 1.5: Traceability from objectives to implementation and evidence....8
Table 2.1: Summary of literature survey 13
Table 2.2: Comparison of token and session-management designs....15
Table 2.3: Established prior work and what this project adds 16
Table 3.1: Packages in the Ninsho monorepo (release 0.1.1) 18
Table 3.2: Opaque and PASETO v4.public strategies compared (from repository documentation) 19
Table 3.3: Test data and experimental inputs 21
Table 3.4: Assets protected by Ninsho....27
Table 3.5: Adversaries, capabilities and primary controls....28
Table 3.6: Configuration options of the Ninsho constructor 36
Table 3.7: Store key schema (prefix ninsho:v1:) 37
Table 3.8: Error classes, codes and HTTP statuses (packages/core/src/errors.ts) 38
Table 3.9: Responsibilities of Ninsho and of the integrating application....38
Table 3.10: What the SecureShop comparisons demonstrate and what they do not establish....40
Table 4.1: Evidence categories and their status 41
Table 4.2: Experimental environments recorded in the repository....42
Table 4.3: Security behaviours and the test files that evidence them 43
Table 4.4: Manual attack procedures grouped by theme (project-recorded, release 0.1.0) 44
Table 4.5: Browser regression results (project-recorded local run on Windows, Node 22.13.1, Playwright 1.64.1)....45
Table 4.6: Redis scenarios and recorded outcomes (project-recorded; Docker-managed tests) 46
Table 4.7: SecureShop documented outcomes by comparison....47
Table 4.8: Organization of the automated tests (fresh run, 21 October 2026)....48
Table 4.9: Selected test cases with setup, expected result and recorded outcome....49

Table 4.10: Benchmark configurations and the checks each performs per request 51

Table 4.11: Verification cost at one operation at a time (milliseconds) 52

Table 4.12: Protected-request throughput (successful requests per second) 53

Table 4.13: Comparison of the SecureShop baseline with Ninsho configurations (design properties and documented outcomes) 54

Table 4.14: Security objectives, controls, evidence and residual risk 56

Table 4.15: Confirmed findings and their status 56

Table 4.16: Open questions and the evidence that would be needed 58

Table 5.1: Objectives and the evidence supporting them 60

Table 5.2: Prioritized future work 63

Abbreviations

AOF – Append-Only File
API – Application Programming Interface
BOLA – Broken Object Level Authorization
CBOR – Concise Binary Object Representation
CDF – Cumulative Distribution Function
CI – Continuous Integration
COSE – CBOR Object Signing and Encryption
CSPRNG – Cryptographically Secure Pseudo-Random Number Generator
DoS – Denial of Service
CV – Coefficient of Variation
DPoP – Demonstrating Proof of Possession
FIDO – Fast IDentity Online
FR – Functional Requirement
HTTP – Hypertext Transfer Protocol
HTTPS – Hypertext Transfer Protocol Secure
IDOR – Insecure Direct Object Reference
IETF – Internet Engineering Task Force
JSON – JavaScript Object Notation
JWK – JSON Web Key
JWS – JSON Web Signature
JWT – JSON Web Token
MDS – Metadata Service (FIDO)
NFR – Non-Functional Requirement
OWASP – Open Worldwide Application Security Project
PAE – Pre-Authentication Encoding
PASETO – Platform-Agnostic Security Tokens
RBAC – Role-Based Access Control
RDB – Redis Database snapshot
RFC – Request for Comments
RP – Relying Party
SDK – Software Development Kit
TB – Trust Boundary
TTL – Time To Live

W3C – World Wide Web Consortium
WebAuthn – Web Authentication
WSL – Windows Subsystem for Linux
XSS – Cross-Site Scripting

Chapter 1

Introduction

Almost every networked application must answer two questions on every request: who is making it, and may that caller do what is being asked. The first is authentication and the second is authorization. Once a user has proved their identity, usually by a password or a passkey, the application issues a credential that stands in for that proof on later requests. How that credential is created, protected, renewed and withdrawn is called session management. It is a small part of an application's code and a large part of its attack surface.

This report describes the design and evaluation of **Ninsho**, an authentication and session-management software development kit (SDK) for Node.js. Ninsho does not verify passwords or federated sign-in; it takes over after identity is established and handles issuing, verifying, rotating and revoking session credentials, answering authorization questions, limiting request rates, and, optionally, verifying WebAuthn passkey ceremonies. The source code is public [1] and four packages are published on the npm registry under the scope `@ninshorg`.

1.1 Background

A *bearer token* is a credential that grants access to whoever presents it, without any proof that the presenter is the party to whom it was issued [2]. Bearer tokens are simple and widely deployed, but a token copied from a log file, a browser store or a network trace is as useful to an attacker as to its owner. Two designs dominate practice. In the first, the token is an opaque random string and the server looks it up in a store on each request; withdrawing a session means deleting a record. In the second, the token is self-contained and signed, as with a JSON Web Token (JWT) [3]; the server verifies the signature and reads the claims without consulting any store. The second design scales easily across servers, but a signed token remains acceptable until it expires unless the server adds state, such as a deny-list, a per-user session version or token introspection, to withdraw it earlier.

Short-lived access tokens are usually paired with longer-lived *refresh tokens*. Current OAuth 2.0 security guidance recommends that refresh tokens either be bound to the client

by sender-constraining or be rotated on use, so that presentation of a token that was already used signals theft [4]. *Sender-constrained* tokens can be achieved with Demonstrating Proof of Possession (DPoP), in which the client holds a private key and signs a small proof for every request, so that a stolen token without the key is not enough [5]. For passwordless sign-in, the W3C Web Authentication (WebAuthn) specification defines how a relying party registers and verifies public-key credentials held by authenticators [6].

Authentication alone does not protect user data. Broken object-level authorization, where a valid user requests another user's object by changing an identifier, tops the OWASP API Security list [7]. Libraries therefore need to make ownership checks easy to apply correctly. Implementation quality matters as well: a recent large-scale study of JWT libraries found previously unknown vulnerabilities in a substantial share of the implementations tested [8], and the IETF has published best-practice guidance for using JWTs safely [9].

1.1.1 Identity verification, authentication, authorization and session management

Four activities are often grouped under the word authentication, and separating them clarifies what Ninsho does and does not do. *Identity verification* establishes that a real person or organization corresponds to an account, for example by checking a document or an email address. *Authentication* establishes that the party making a request is the holder of an account, typically by a password, a passkey or a federated sign-in. *Authorization* decides whether that authenticated party may perform the requested operation on the requested resource. *Session management* covers everything between a successful authentication and the end of the session: issuing a credential that stands in for the authentication, checking it on later requests, renewing it, and withdrawing it at logout or when it is suspected to be stolen.

Ninsho deliberately implements only the last two. The host application verifies passwords or passkey assertions and owns the user records; once it is satisfied, it calls `createSession()` with a *principal* (a user identifier, roles, scopes and an optional tenant) and Ninsho takes over the session lifecycle and the route-level authorization checks. The optional browser client handles how the browser holds and presents credentials, and the optional WebAuthn package assists the application with passkey authentication without taking over the user records. Identity verification is outside the scope of this project.

1.1.2 Access tokens, refresh tokens and their lifetimes

An *access token* accompanies every request to a protected route and should be short-lived, because it is the credential most often exposed in logs, browser memory and network traces. A *refresh token* is presented only to a refresh endpoint and is used to obtain a new access token when the current one expires, so that the user does not have to authenticate again every few minutes. In Ninsho the defaults are an access-token lifetime of 300 seconds and a refresh-token lifetime of 604,800 seconds (seven days). The refresh lifetime is also the absolute lifetime of the *token family*: every refresh token produced by rotation inherits the family's fixed expiry, so a session cannot be extended indefinitely by refreshing it.

Each refresh token is single-use. Presenting it consumes it and returns a replacement with the next *generation* number; the old token is recorded as consumed by a short-lived *tombstone*. This rotation is what allows a stolen refresh token to be detected: if both the legitimate client and an attacker hold the same token, the second presentation finds the tombstone and reveals the theft [4].

1.1.3 Practical problems in session lifecycle management

The difficulties that motivated Ninsho are not exotic cryptographic attacks but ordinary operational situations in which a straightforward implementation behaves incorrectly. Table 1.1 lists the situations that recur in this report, the consequence of handling each naively, and the mechanism Ninsho uses. Each mechanism is described in Chapter 3 and its evidence is reported in Chapter 4.

Table 1.1: Session-management problems and the corresponding Ninsho mechanisms

Situation	Naive consequence	Ninsho mechanism
User logs out while a copy of the access token exists elsewhere	Copied token keeps working until expiry	Opaque record deleted (or PASETO deny-list entry written) at logout; next verification fails
Several application instances behind a load balancer	Logout on one instance not visible to another	All state in a shared store; every instance reads the same record
Refresh token stolen and used by an attacker	Attacker silently holds a renewable session	Single-use rotation; replay after the grace window revokes the whole family and emits refresh.reuse_detected
Two browser tabs refresh at the same moment	One tab is signed out or reuse is falsely detected	Atomic consume (one winner) and a 30-second grace mapping that hands the loser the same replacement
Bearer access token copied from a log or proxy	Copy is usable by anyone	Optional DPoP binding: every request needs a fresh proof signed by the client's key
Captured DPoP proof replayed	Proof reused within its acceptance window	Proof identifier claimed once with an atomic set-if-absent

Situation	Naive consequence	Ninsho mechanism
Store unreachable during a request	Revocation silently unchecked	Fail closed by default: HTTP 503 instead of an unverified pass
Password changed or account compromised	Old sessions survive	revokeAllForUser() over the user's session index
Signed-in user changes an identifier in the URL	Reads another user's data (BOLA)	requireOwner() refuses when the owner cannot be established or differs
Credential guessing against the login route	Unlimited attempts	Two-dimensional rate limiting by address and account

1.2 Motivation

Developers who need sessions in a Node.js service typically combine a JWT library, a cookie handler, a rate limiter, a Redis client and hand-written middleware. Each piece can be correct while the combination is not: refresh tokens that can be replayed, logout that does not take effect on other server instances, tokens that can be copied from browser storage and used from another machine, and authorization checks forgotten on a route. The motivation for Ninsho is to provide these pieces as one tested library whose documentation names, for every security property it claims, the automated test that demonstrates it.

A second motivation is evidential. Passing tests are easy to over-read. Over the course of the project, a follow-up review reproduced five defects despite a passing suite, including a race between session creation and revocation. This report therefore treats the tests as scoped evidence, separates what was executed from what was only documented, and states limitations explicitly.

1.3 Problem Statement

Applications that issue session credentials need a single component that keeps credential lifecycle consistent across server instances and browser tabs, detects stolen refresh tokens, optionally prevents replay of stolen access tokens, and separates authentication from authorization, while failing safely when its backing store is unavailable. Existing practice either leaves these concerns to ad-hoc combinations of libraries or documents security properties without a traceable test for each. This project designs, implements and evaluates an SDK, Ninsho, that provides these behaviours and reports the evidence for and against each property.

1.4 Objectives

The objectives of this project are:

1. To study standards and research on token-based authentication, session revocation, refresh-token security, proof-of-possession and WebAuthn.
2. To design and implement a Node.js SDK that provides opaque or PASETO access tokens, rotating refresh tokens with reuse detection, optional DPoP binding, authorization guards and an injected store with a Redis implementation.
3. To provide a browser client that holds a non-extractable key, refreshes tokens automatically and invalidates peer tabs on logout.
4. To evaluate the behaviour of the SDK through automated tests, manual attack procedures, browser regressions, Redis restart, outage and data-loss scenarios, and a demonstration application.
5. To compare, within a clearly stated scope, the behaviour of a deliberately limited stateless JWT baseline with that of Ninsho in the demonstration application.
6. To document limitations and unverified areas so that claims remain proportionate to the evidence.

1.5 Scope of the Work

The work covers the design, the implementation as it stands at repository commit 6f82026 (release v0.1.1), and the evidence recorded in the repository and in public CI. The following are within scope:

- opaque access tokens, session creation, refresh-token rotation, reuse detection and revocation;
- optional PASETO v4.public access tokens with a store-backed deny-list;
- optional DPoP binding with proof validation and replay protection, and the browser client with cross-tab invalidation;
- authorization guards, rate limiting and framework adapters for Express-style, Fastify, Hono and Koa applications;
- the optional WebAuthn ceremony and attestation verifier;
- Redis persistence, outage and data-loss behaviour; and

- the SecureShop demonstration application.

The following are outside the scope and are not claimed: password storage or verification, federated sign-in, a user study or any deployment in production, certification, formal verification, physical-device interoperability testing for passkeys, and a performance comparison with JWT libraries. Where the repository records measurements, they are reported as historical measurements of the stated version and machine, not as properties of the current code.

1.6 Intended Users, Assumptions and Constraints

The intended users of Ninsho are developers of Node.js web services and APIs who need server-side session management, and who are able to run a Redis instance shared by their application servers. The library is also used in this project by the students themselves for the evaluation and by examiners through the SecureShop demonstration. End users of an application interact with Ninsho only indirectly, through the browser client and the application's own pages.

The design and the evaluation rest on the assumptions and constraints in Table 1.2. Where an assumption does not hold, the corresponding guarantee does not hold either; Chapter 4 records the cases that were tested and those that were not.

Table 1.2: Assumptions and constraints

Type	Statement	Consequence if violated
Assumption	The application authenticates users correctly before calling createSession()	Ninsho would issue sessions to the wrong principal; it cannot detect this
Assumption	All application instances share one Redis deployment and the same configuration	Revocation on one instance would not be visible to another
Assumption	Redis provides atomic GETDEL and SET NX EX (Redis 6.2 or later)	Two concurrent callers could both rotate a refresh token or reuse a proof
Assumption	The deployment is served over HTTPS	Tokens and cookies could be observed in transit
Assumption	A reverse proxy, if present, sets Host and X-Forwarded-* reliably, and trustProxy is configured to match	DPoP URI binding and per-address rate limits can be influenced by the client
Constraint	Node.js 20 or later; npm 11 for the workspace tooling	Older runtimes are not supported or tested
Constraint	No credential verification, user database or federated sign-in inside Ninsho	These remain application responsibilities
Constraint	core, client and webauthn have no third-party runtime dependencies (enforced in CI)	Adding one fails the bundle-purity job
Constraint	Evaluation on one Windows laptop with student resources	Results are machine-specific and exploratory

1.7 Requirements

The requirements below were derived from the objectives in Section 1.4 and from the security properties stated in the repository's SECURITY.md. Table 1.3 lists the functional requirements and Table 1.4 the non-functional requirements. Priorities use the MoSCoW convention (Must, Should, Could). The acceptance criterion of each requirement names the kind of evidence that would show it is met, and Section 1.8 links the requirements to the implementation and the recorded evidence.

1.7.1 Functional requirements

Table 1.3: Functional requirements

ID	Requirement	Priority	Acceptance criterion
FR-01	Create a session for an application-supplied principal and return an access token and a refresh token	Must	Session creation tests; token pair returned
FR-02	Verify an access token on each request and expose the caller's identity to route handlers	Must	Middleware tests; 401 for missing, invalid, expired or revoked tokens
FR-03	Rotate refresh tokens on every use and limit the family to a fixed absolute lifetime	Must	Rotation tests; family expiry never extended
FR-04	Detect reuse of a rotated refresh token and revoke the whole family	Must	Reuse tests; refresh.reuse_detected emitted
FR-05	Revoke one session, or all sessions of a user, with effect on every instance sharing the store	Must	Revocation and concurrency tests against Redis
FR-06	Offer PASETO v4.public access tokens with issuer, audience and deny-list checks as an option	Should	Official test vectors; engine tests
FR-07	Bind tokens to a client key with DPoP and reject replayed proofs	Should	DPoP proof, request and integration tests
FR-08	Provide role, scope, ownership, tenant and fresh-authentication guards	Must	Middleware and fresh-authentication tests
FR-09	Rate-limit endpoints by client address and by account identifier	Should	Rate-limiter tests
FR-10	Issue and consume single-use tokens for password reset and similar flows	Should	One-time-token tests, including concurrent consumption
FR-11	Provide a browser client that refreshes automatically and invalidates peer tabs on logout	Should	Client tests; browser regressions on three engines
FR-12	Verify WebAuthn registration and authentication ceremonies as an optional package	Could	WebAuthn package tests, attestation fixtures

1.7.2 Non-functional requirements

Table 1.4: Non-functional requirements

ID	Category	Requirement	How it is checked
NFR-01	Security	The shortest configuration must be the safest: opaque tokens, fail-closed, five-minute access tokens	config.test.ts secure-defaults tests
NFR-02	Security	Raw access and refresh tokens are never stored; only SHA-256 hashes	opaque-engine tests (raw token never stored)

ID	Category	Requirement	How it is checked
NFR-03	Reliability	When the store is unreachable, no request is admitted without its revocation check unless fail-open is chosen explicitly	store-invariants and persistence tests
NFR-04	Correctness under concurrency	Of concurrent callers presenting one single-use credential, exactly one succeeds	store contract and concurrency tests against Redis
NFR-05	Portability	No framework dependency; adapters for Express-style, Fastify, Hono and Koa; ESM and CommonJS builds	Adapter tests; bundle-purity CI job
NFR-06	Supply chain	core, client and webauthn ship with no third-party runtime dependencies	Bundle-purity CI job; consumer dependency audit
NFR-07	Observability	Security-relevant events are emitted to an injectable audit sink	Audit-event assertions in tests
NFR-08	Maintainability	Strict TypeScript; documentation claims must name a test	npm run typecheck; review practice
NFR-09	Performance	Verification cost should be dominated by the single store round trip	Exploratory benchmark (Section 4.2)

1.8 Requirements Traceability

Table 1.5 links each project objective to the requirements it implies, the components that implement them and the evidence available. The evidence column distinguishes automated tests from demonstrations and exploratory measurements; a requirement supported only by design reasoning is marked as such.

Table 1.5: Traceability from objectives to implementation and evidence

Objective	Requirements	Implementing components	Evidence
1. Study standards and research	-	Chapter 2	Literature review; references verified against publisher pages
2. Design and implement the SDK	FR-01 to FR-10, NFR-01 to NFR-08	packages/core, packages/server	2,150 tests at commit 6f82026 with real Redis; CI run on the same commit
3. Browser client	FR-11	packages/client	Client unit tests; six browser regression runs (three engines)
4. Evaluate by tests, attacks, browsers, Redis and demonstration	NFR-03, NFR-04	Test suites, manualtest/, demo/viva-showcase	Fresh re-run (21 Oct 2026); recorded manual attacks (release 0.1.0); SecureShop tests
5. Compare with a JWT baseline	NFR-09	research/validation/benchmarks	Exploratory three-trial benchmark; SecureShop outcomes
6. Document limitations	-	SECURITY.md, Chapter 4	Findings and limitations sections
Optional passkey support	FR-12	packages/webauthn	623 WebAuthn tests at commit 6f82026

1.9 Organization of the Report

The rest of the report is organized as follows. Chapter 2 reviews standards, published research and existing approaches to token and session management, and identifies the engineering gap. Chapter 3 presents the proposed methodology: the architecture, the threat model, the detailed design of each lifecycle operation, the implementation and the SecureShop demonstration. Chapter 4 reports the results, separating recorded evidence from unexecuted checks, and evaluates performance and security. Chapter 5 concludes the report and sets out prioritized future work, and Chapter 6 lists publications and competitions.

Chapter 2

Literature Survey

This chapter reviews standards and research relevant to the design decisions in Ninsho. Because authentication libraries implement specifications, the standards are reviewed together with the peer-reviewed work. Peer-reviewed sources were selected for relevance to web authentication, session security and token implementations; the review does not claim to be an exhaustive survey of the field.

2.1 Review of Existing Work

2.1.1 Authentication schemes and web sessions

Bonneau et al. [10] proposed a framework for comparing web authentication schemes against usability, deployability and security benefits, and used it to compare password-replacement proposals. Their central lesson for library design is that schemes involve trade-offs rather than a single dominant choice, which motivates Ninsho's decision to make stronger options, such as DPoP binding and PASETO tokens, optional and to document their costs. Squarcina et al. [11] studied how weaknesses in cookie handling undermine the integrity of web sessions and proposed defences. Ninsho's example application keeps the refresh token in an HTTP-only cookie, so cookie-layer integrity remains the responsibility of the host application and its deployment, a limitation that is stated in Chapter 5.

2.1.2 OAuth, token theft and refresh-token security

Fett, Küsters and Schmitz [12] analysed OAuth 2.0 in a formal model of the web infrastructure, reported previously unknown attacks and proposed fixes. The IETF OAuth Security Best Current Practice [4] consolidates such findings. Among its recommendations for refresh tokens are sender-constraining and rotation with reuse detection, which Ninsho implements as single-use refresh tokens, a short grace window for concurrent tabs, and revocation of the whole token family when a consumed token is replayed after the grace window. The base framework and bearer-token usage are specified in RFC 6749 [13] and RFC 6750 [2].

2.1.3 JWT security and implementation quality

RFC 7519 [3] defines the JWT format, and RFC 8725 [9] collects best current practices for its use, including strict algorithm handling and validation of claims. Yang et al. [8] developed a testing methodology, JWTeemo, and applied it to 43 JWT implementations in ten programming languages; they reported 31 previously unknown vulnerabilities, 20 of which received CVE identifiers, with authentication bypass in Kubernetes and denial of service against Apache James among the demonstrated impacts. This supports a conservative approach to token parsing. Ninsho's default token is an opaque random string with no parsing surface, and its optional PASETO implementation is written to the specification with a fixed algorithm, strict base64url canonicality checks, and the official test vectors [14]. The study concerns implementation bugs; it does not argue that JWTs cannot be used securely, and neither does this report.

2.1.4 Proof-of-possession and sender-constrained tokens

RFC 9449 [5] specifies DPoP, in which the client proves possession of a private key with a signed proof bound to the HTTP method, the target URI, a unique identifier, a time of issue and, for resource requests, a hash of the access token. The specification recommends that servers track proof identifiers to limit replay within the acceptance window. JWK thumbprints used to bind a token to a key follow RFC 7638 [15]. Ninsho's server verifies these elements and claims each proof identifier atomically in the store; its browser client generates a non-extractable WebCrypto key and persists it in IndexedDB.

2.1.5 Passkeys and WebAuthn

The WebAuthn specification [6], which became a W3C Recommendation on 25 August 2026, defines registration and authentication ceremonies, attestation formats and signature counters. It relies on CBOR [16] and on COSE keys. Lassak et al. [17] studied the obstacles that companies face when deploying FIDO2 passwordless authentication; their focus is organizational deployment rather than the implementation of a verifier. Ninsho's optional WebAuthn package verifies ceremonies and returns a principal, leaving credential storage and user records to the application.

2.1.6 Server-side sessions, opaque tokens and introspection

The oldest design keeps all session state on the server and gives the client only a random identifier, usually in a cookie. The OAuth 2.0 framework [13] leaves the format of access tokens to the authorization server, and OAuth token introspection [21] standardizes how a

resource server can ask the issuer whether an opaque token is active. The cost of this design is a lookup on every request; the benefit is that revocation, expiry and the token's meaning are all decided by the server at the moment of use. Ninsho's default *opaque* strategy follows this design but performs the lookup directly against the shared store rather than through an introspection endpoint, and it stores only a SHA-256 hash of each token, so that read access to the store does not yield usable credentials.

Calzavara et al. [22] surveyed attacks on web sessions, including session hijacking and fixation, and the defences proposed for them, and compared those defences on protection, usability, compatibility and ease of deployment. Their survey frames the session as a sequence of states that an attacker may try to enter, which is the view adopted in Chapter 3's lifecycle design.

2.1.7 PASETO and the removal of algorithm agility

Many JWT vulnerabilities stem from the token itself naming the algorithm used to verify it, which allows an attacker to choose a weaker or different algorithm [9]. PASETO version 4 removes this negotiation: the version and purpose prefix `v4.public.` fixes the algorithm to Ed25519 [18], and the header, payload, footer and an optional implicit assertion are bound together by pre-authentication encoding before signing [14]. Ninsho's optional PASETO strategy implements v4.public with a key identifier in the footer, requires issuer and audience, and checks the official test vectors. A signed token can be verified without the store, which is useful when several services verify independently, but revocation then needs a deny-list held in the store, which reintroduces the lookup it was meant to avoid.

2.1.8 Revocation strategies and their consistency trade-offs

OAuth token revocation [20] defines how a client tells the authorization server that a token is no longer needed; it does not define how every resource server learns of the revocation. The practical strategies differ in how quickly a revocation takes effect, what state they require and how they fail when that state is lost. Relying on a short lifetime alone needs no state but leaves a revoked token usable until it expires. A deny-list of revoked identifiers takes effect immediately but must be kept until each token expires, and if it is lost the revoked tokens become valid again. A per-user version number carried in each token revokes all of a user's tokens with one counter but cannot revoke a single session. Introspection asks the issuer on every request and fails closed if the issuer is unreachable. A reference (opaque) token is itself the lookup key, so revocation is deletion and losing the

store ends every session. Ninsho's default uses the last of these and its PASETO strategy the deny-list; the difference in failure behaviour is the property this project examines most closely in Chapter 4.

2.1.9 Session hijacking and consistency across tabs and devices

Sivakorn et al. [23] showed that session cookies sent over unencrypted connections allowed attackers to hijack accounts on major services and, in some cases, to bypass additional authentication requirements. Squarcina et al. [11] later showed that even with modern cookie attributes, weaknesses in how cookies are scoped and overwritten allow session integrity attacks. Both results argue for limiting what a stolen credential can do, which is the purpose of sender-constraining with DPoP [5], and for keeping the long-lived credential in an HTTP-only cookie rather than in script-readable storage.

Consistency across browser tabs is a client-side problem that the reviewed specifications do not address. When one tab logs out, other tabs of the same origin may still hold an access token in memory and may be in the middle of a refresh. Ninsho's client uses the BroadcastChannel API to send credential-free notices so that peer tabs discard their in-memory state; the server remains authoritative, and a tab that misses a notice learns of the logout on its next request.

2.2 Summary of Literature

Table 2.1 summarizes the literature and specifications reviewed. The column headed "Dataset / Scope" records the subject of each work, since most are standards or analytical studies without datasets.

Table 2.1: Summary of literature survey

Sr. No.	Author(s) & Year	Method / Technique	Dataset / Scope	Key Result / Contribution	Limitation
1	Bonneau et al., 2012 [10]	Comparative evaluation framework	Web authentication schemes (analytical)	Structured comparison; shows trade-offs between schemes	Predates widespread passkeys; not about token lifecycles
2	Fett et al., 2016 [12]	Formal analysis in a web infrastructure model	OAuth 2.0 flows (formal model)	Reported previously unknown attacks and proposed fixes	Models protocol flows, not a particular SDK or store
3	Squarcina et al., 2023 [11]	Analysis of cookie-based session integrity	Web session cookies	Attacks on and defences for session integrity	Cookie layer only; token lifecycle out of scope
4	Lassak et al., 2024 [17]	Study of FIDO2 deployment obstacles	Companies deploying passkeys	Identifies deployment obstacles	Organizational view; no implementation evaluation

Sr. No.	Author(s) & Year	Method / Technique	Dataset / Scope	Key Result / Contribution	Limitation
5	Yang et al., 2026 [8]	Testing methodology (JWTeemo)	43 JWT libraries, 10 languages	31 new vulnerabilities, 20 CVEs	Implementation bugs; no revocation-design comparison
6	IETF, RFC 8725, 2020 [9]	Best current practice	JWT usage	Practices for safe JWT handling	Guidance, not measured evidence
7	IETF, RFC 9449, 2023 [5]	Standards-track specification	Sender-constrained OAuth tokens	DPoP proofs and replay recommendations	Needs per-request signing and key management
8	IETF, RFC 9700, 2025 [4]	Best current practice	OAuth 2.0 deployments	Rotation and sender-constraining guidance	Guidance for OAuth; not an implementation
9	W3C, WebAuthn L3, 2026 [6]	Recommendation	Passkey ceremonies	Registration, assertion and attestation rules	Leaves storage and sessions to the relying party
10	PASETO Standard [14]	Community specification	Signed and encrypted tokens	Alternative to JOSE tokens	Maintained outside the IETF RFC series
11	Calzavara et al., 2017 [22]	Survey and classification	Web session attacks and defences	Comparison of defences on protection, usability, compatibility and deployment	Survey; no implementation evaluated
12	Sivakorn et al., 2016 [23]	Empirical study of cookie hijacking	Major web services	Showed hijacked cookies expose private data and can bypass protections	Concerns unencrypted transport; token lifecycle out of scope
13	IETF, RFC 7009, 2013 [20]	Standards-track specification	OAuth token revocation	Client-initiated revocation endpoint	Does not define propagation to resource servers
14	IETF, RFC 7662, 2015 [21]	Standards-track specification	OAuth token introspection	Resource servers can query token state	Adds a network call per check; issuer must be reachable

2.3 Research Gaps

Within the sources reviewed, the following gaps motivate this project. They are statements about the reviewed material, not claims about all prior work.

- **Integrated, runnable behaviour.** Standards and studies describe rotation, sender-constraining and revocation separately; they do not provide one small, runnable library in which these behave consistently and can be exercised together.
- **Failure behaviour under store loss.** The reviewed sources say little about what happens to revocation when the backing store is restarted without data, rolled back or unavailable. Ninsho documents and tests these cases for two token strategies.
- **Browser lifecycle races.** Guidance concentrates on the server. Stale refresh responses, concurrent key creation across tabs and cross-tab logout are client-side concerns that the reviewed sources do not address directly.

- **Traceability of security claims.** Security documentation commonly lists properties without naming the test that demonstrates each. This project adopts a claim-to-test mapping and applies it to its own documentation, including correcting overstated claims.
- **Honest comparison baselines.** Comparisons of stateless tokens against sessions often use a deliberately weak baseline. This project states the baseline's limits and does not claim that JWT systems cannot implement revocation or sender-constraining.

2.4 Comparison of Token and Session-Management Designs

Table 2.2 compares four representative designs on the properties examined in this project. It describes what each design provides by itself; any of them can be extended, and a JWT-based system can add a deny-list, introspection or DPoP. The comparison therefore explains design choices rather than ranking technologies.

Table 2.2: Comparison of token and session-management designs

Property	Stateless JWT	JWT with shared deny-list	Opaque token with shared store	Opaque token with DPoP
Server state per request	None	One lookup	One lookup	One lookup and one proof claim
Revocation before expiry	No	Yes, while the deny-list is available	Yes	Yes
Copied access token usable elsewhere	Yes, until expiry	Yes, until revoked	Yes, until revoked	No, without the client's key
Claims readable by the holder	Yes	Yes	No	No
Algorithm negotiation in the token	Yes (alg header)	Yes (alg header)	Not applicable	Proof uses an allow-list (ES256, EdDSA)
Behaviour when the store is lost	Not applicable	Revoked tokens become valid again	All sessions end	All sessions end
Verification without the store	Yes	Signature only	No	No

2.5 Positioning of Ninsho Relative to Prior Work

None of the mechanisms used by Ninsho is new: opaque tokens, rotation with reuse detection, deny-lists, PASETO, DPoP and WebAuthn are all specified or published by others, and the cryptographic primitives come from Node.js's own crypto module. What the project contributes is an engineering composition with stated defaults and evidence, summarised in Table 2.3. These are claims about this implementation within the reviewed

literature, not claims of priority over all existing libraries, since no systematic survey of Node.js session libraries was carried out.

Table 2.3: Established prior work and what this project adds

Aspect	Established by prior work	What this project adds
Refresh rotation and reuse detection	RFC 9700 recommendation [4]	A store-ordering design that keeps concurrent tabs signed in, with regression tests against real Redis
Sender-constrained tokens	RFC 9449 DPoP [5]	Binding checked before the refresh token is consumed, so a thief without the key cannot destroy the session
Revocation	RFC 7009, introspection, deny-lists [20], [21]	Opaque default with immediate revocation, and measured documentation of PASETO revival after store loss
Failure behaviour	General fail-safe design principle	Fail-closed default, and refusal at configuration time of fail-open combinations that cannot work
Authorization	OWASP API1, broken object-level authorization [7]	Guards that refuse when they cannot decide, including Express 5 array parameters
Evidence practice	-	Each documented security property names its test; evidence is labelled by source and commit

Chapter 3

Proposed Methodology

This chapter describes the system, the inputs used to exercise it, the algorithms that implement the token lifecycle, the evaluation metrics and the tools. Ninsho is a software library and not a learning system, so there is no training dataset and no classification accuracy, precision, recall or F1-score in this report. Evaluation is by whether specified behaviours occur, expressed as test outcomes and observed HTTP status codes, and by latency and throughput measurements where the repository records them.

3.1 System Architecture

The block diagram of the proposed system is shown in Figure 3.1. Ninsho is a monorepo of four packages. The application sits between an optional browser client and the Ninsho server package, and Ninsho reads and writes all state through a small store interface.

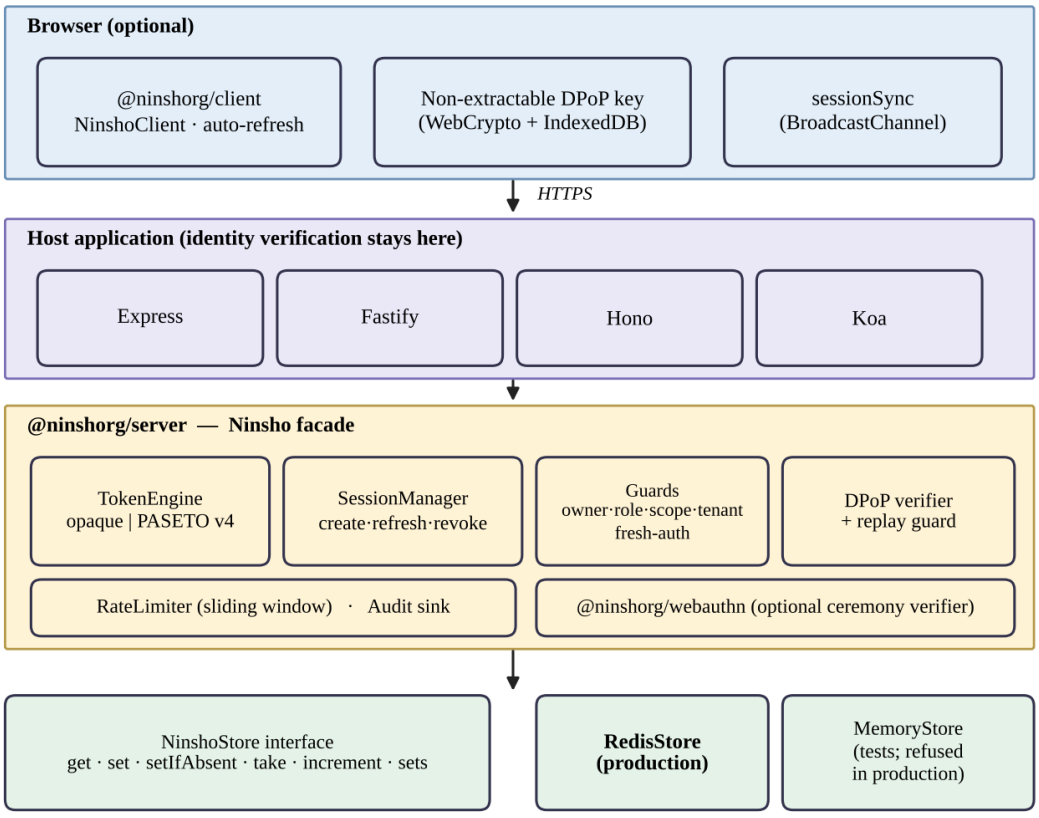


Figure 3.1: Block diagram of the Ninsho architecture

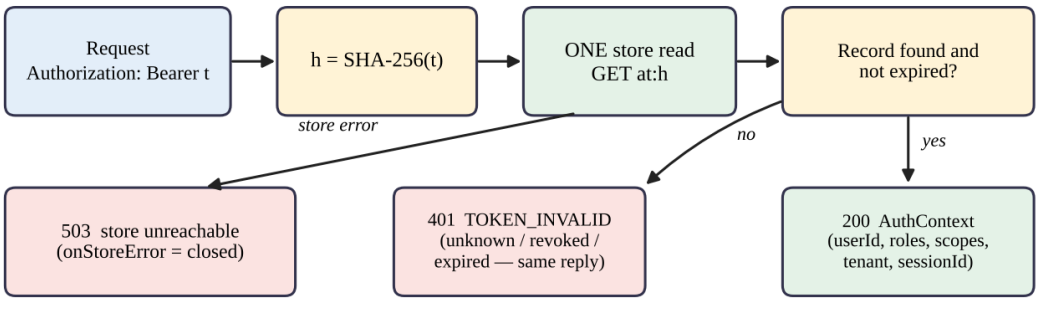
Table 3.1 lists the packages. The server package exposes a facade, `Ninsho`, over a token engine, a session manager, middleware guards, a rate limiter and the DPoP verifier. Each piece is also exported separately, so an application can assemble them differently.

Table 3.1: Packages in the Ninsho monorepo (release 0.1.1)

Package	Role	Runtime dependencies
`@ninshorg/core`	Shared types, errors and cryptographic primitives (token generation, SHA-256 hashing, constant-time comparison)	None
`@ninshorg/server`	Sessions, token engines (opaque and PASETO v4.public), guards, rate limiting, DPoP verification, stores, adapters for Fastify, Hono and Koa	`ioredis`, `@ninshorg/core`
`@ninshorg/client`	Browser client: DPoP key management, automatic refresh, cross-tab invalidation	None
`@ninshorg/webauthn`	WebAuthn registration and authentication verification, attestation formats, test authenticator (separate subpath)	`@ninshorg/core` only

3.1.1 Token engines and the one-read verification path

The default engine issues opaque tokens: 32 bytes from a cryptographically secure generator, encoded in base64url. The server stores only the SHA-256 hash of the token, so a leaked store does not reveal usable tokens. Verification hashes the presented token and performs one store read, as shown in Figure 3.2. Because revocation deletes the record, the same read that authenticates a request also enforces revocation. If the store is unreachable the default policy fails closed with HTTP 503, and for the opaque engine the configuration that would fail open is refused at start-up, because the store is the token.



Revocation deletes the record, so the same single read that authenticates also enforces revocation.

Figure 3.2: Opaque token verification: one store read per request

The optional engine issues PASETO v4.public tokens, which are signed with Ed25519 [18] and verified locally with a key identified by the token footer [14]. Signature verification

needs no store read, but revocation requires a deny-list in the store, and this difference drives the trade-offs summarized in Table 3.2.

Table 3.2: Opaque and PASETO v4.public strategies compared (from repository documentation)

Aspect	Opaque (default)	PASETO v4.public (optional)
Token content	Random lookup key; claims stay on the server	Signed claims readable by the holder
Verification	SHA-256 and one store read	Ed25519 signature check, plus deny-list lookup if revocation is enforced
Revocation	Delete the record; immediate	Deny-list entry in the store until expiry
Store outage, fail-closed	Rejects (503)	Rejects (503)
Store outage, fail-open	Refused at configuration time	Accepts valid signatures without a deny-list check; revoked tokens work during the outage
Complete store data loss	All tokens unknown; users must sign in again	Active tokens keep verifying; previously revoked tokens verify again until expiry

3.1.2 Session manager, refresh tokens and the store

The session manager creates a session, rotates refresh tokens and revokes sessions. A session has a fixed absolute lifetime set at creation, which rotation cannot extend. Defaults are an access-token lifetime of 300 seconds, a refresh-token lifetime of seven days, a refresh grace window of 30 seconds and a clock tolerance of five seconds. All state is read and written through the `NinshoStore` interface, whose operations include `get`, `set`, `setIfAbsent`, an atomic `take` (read and delete), `increment` and set operations. The production implementation, `RedisStore`, requires Redis 6.2 or later because `GETDEL` makes `take` atomic; commands issued during an outage are rejected rather than queued. `MemoryStore` is intended for tests and refuses to be constructed when `NODE_ENV` is `production`.

3.1.3 Authorization guards and framework integration

Authentication answers who the caller is; authorization answers what the caller may do. After the verification middleware has populated the request context, six guards can be mounted: `requireRole`, `requireAllRoles`, `requireScope`, `requireOwner`, `requireTenant` and `requireFreshAuth`. A guard refuses rather than guesses when it cannot decide; for example, an ownership selector that resolves to nothing yields a 403 and not a pass. This addresses the broken object-level authorization risk [7], but the application must still supply

a trustworthy owner for each resource. Adapters for Fastify, Hono and Koa are published as separate subpath exports with no runtime framework dependency.

3.1.4 Browser client and cross-tab invalidation

The browser client keeps the access token in memory, generates a non-extractable WebCrypto key and stores it in IndexedDB, refreshes automatically and attaches DPoP proofs when binding is enabled. Key creation uses a single read-write IndexedDB transaction so that tabs initializing concurrently converge on one key. An opt-in `sessionSync` feature broadcasts credential-free notices over the browser's `BroadcastChannel`, as in Figure 3.3. Notices carry only a version, a context name, an event type and a random identifier; they invalidate peer tabs and never transfer a token. Delivery is best-effort and the server remains authoritative.

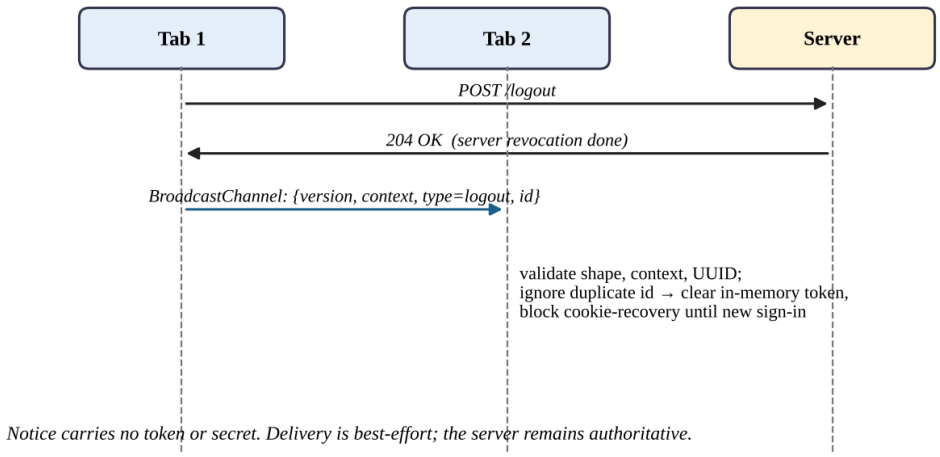


Figure 3.3: Cross-tab invalidation after a confirmed logout

3.1.5 WebAuthn verification

The optional `@ninshorg/webauthn` package verifies the registration and authentication ceremonies: the challenge, the origin, the relying-party identifier, the signature and the signature counter. It parses CBOR with definite lengths only, handles COSE keys, and verifies the attestation formats `none`, `packed`, `apple`, `tpm`, `fido-u2f`, `android-key` and `android-safetynet`, with an optional metadata-service module. It stores no credentials; it returns a principal to the application. A virtual authenticator for tests is exported from a separate subpath and refuses to run under `NODE_ENV=production`.

3.2 Test Data and Experimental Inputs

Ninsho is evaluated with constructed inputs and not with a dataset. Table 3.3 lists the kinds of input used. None of the inputs involves real users or real personal data.

Table 3.3: Test data and experimental inputs

Parameter	Details
Nature of data	Synthetic principals, tokens, DPoP proofs, WebAuthn ceremonies and HTTP requests; no learning dataset
Specification vectors	Official PASETO v4.public vectors 4-S-1 to 4-S-3; the RFC 7638 thumbprint vector; RFC 8949 Appendix A CBOR vectors (as named in repository tests)
Adversarial inputs	Tampered, truncated and malformed tokens; a PASETO review test with 10,000 malformed or adversarial inputs and single-byte mutations of every signature byte; fuzz tests of attacker-controlled parsers
Authenticator fixtures	A virtual authenticator and generated certificates for attestation formats
Application fixtures	Express example and playground applications; manual attack scripts MT-01 to MT-20; SecureShop with two seeded demonstration accounts (an administrator and a customer)
Backing stores	`MemoryStore`; real Redis 7 instances, including disposable Docker instances for persistence tests

3.3 Algorithm Description

The algorithms below summarize the behaviour implemented in `packages/server/src/session/manager.ts`, the token engines and the DPoP module, as read at commit 6f82026. They are descriptions of the implementation and not new algorithms.

3.3.1 Session creation and request verification

1. The application authenticates the user by its own means and calls session creation with a principal.
2. The manager generates a session identifier and a refresh token, stores the refresh record under the token’s SHA-256 hash, and fixes the session’s absolute expiry.
3. It writes session metadata and adds the session to the user’s session index.
4. The engine issues an access token and stores its record under the hash of the token (see Equation (3.1)).
5. The manager checks whether a revocation marker for the session already exists. If so, it revokes the access token and the family and raises an error, so that a credential issued after a completed revocation is never returned.

6. On each request, verification hashes the presented token, reads the record once, checks expiry and DPoP binding, and returns the request context or a 401 or 503 response.

$$h = \text{SHA-256}(t), \quad t = 32 \text{ random bytes from a CSPRNG, base64url-encoded} \qquad (3.1)$$

3.3.2 Refresh-token rotation and reuse detection

1. The client presents a refresh token. The manager peeks at its record and, if the token is DPoP-bound, rejects a missing or different key without consuming the token, so that a thief holding only the token cannot destroy the legitimate session.
2. The manager atomically takes the live record. Exactly one concurrent caller wins.
3. The winner writes a replacement token with the next generation number, a grace mapping from the old hash to the new token for 30 seconds, and a tombstone for the old hash, then issues a new access token. After each step it checks for a revocation marker and undoes its writes if one has appeared.
4. A caller that finds no live record looks for the tombstone. If it is found and a grace mapping exists, the caller adopts the same replacement token, so that parallel tabs converge on one chain.
5. If a tombstone exists but the grace window has passed, the token has been replayed: the whole family is revoked, a reuse-detected audit event is emitted, and a reuse error is returned.
6. If neither record exists, the token is rejected as invalid without revealing why.

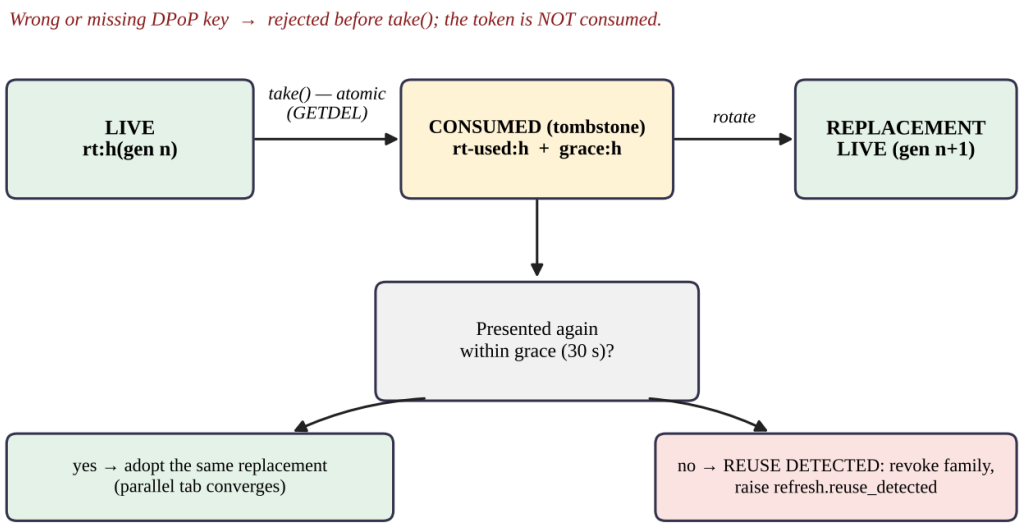


Figure 3.4: Refresh-token states, grace window and reuse detection

3.3.3 Revocation

1. Write a session-revoked marker first, so that concurrent issuance paths can see it.
2. Mark and delete every refresh token, tombstone and grace record in the session's family.
3. Ask the engine to revoke every access token of the session.
4. Remove the session from the user's index. For logout-all, work from one snapshot of the user's indexed sessions, retain any session whose revocation failed, and raise an incomplete-revocation error so the caller can retry.

3.3.4 DPoP proof verification

Figure 3.5 shows the checks that a DPoP-bound request passes, in order. The acceptance window is expressed in Equation (3.2), where the proof maximum age is 60 seconds and the clock tolerance is five seconds by default.

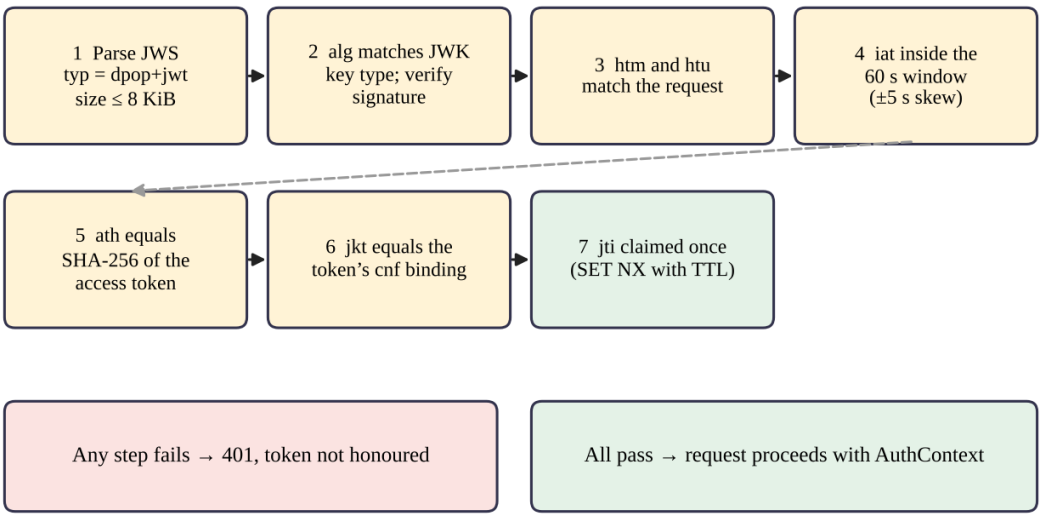


Figure 3.5: DPoP proof validation pipeline

$$\text{now} - (A + \delta) \;\le\; \text{iat} \;\le\; \text{now} + \delta, \quad A = 60\ \text{s}, \quad \delta = 5\ \text{s} \tag{3.2}$$

Proof identifiers are claimed with an atomic set-if-absent operation, keyed by key thumbprint and identifier, with a retention at least as long as the acceptance window, so that two simultaneous replays cannot both succeed. Under an explicit fail-open configuration the replay check is skipped during a store outage; the signature, method, URI and age checks still apply.

3.3.5 Evaluation Metrics

Because outcomes are mostly binary, the principal metrics are counts. For a set of defined scenarios, conformance is the fraction of applicable scenarios whose observed outcome equals the expected outcome (see Equation (3.3)). It is reported with the counts and is not an accuracy figure for a classifier. For timing measurements recorded in the repository, throughput is operations per second over a run (see Equation (3.4)), reported with mean, 95th and 99th percentile latency.

$$\text{Conformance} = \text{scenarios passed} \div (\text{scenarios run} - \text{not applicable}) \tag{3.3}$$

$$\text{Throughput} = \text{operations completed} \div \text{run time in seconds} \tag{3.4}$$

Further reported quantities are the number of automated tests and skipped tests, the outcome of each browser engine, and the outcome of each Redis scenario. No precision, recall or accuracy measure is reported because no classification is performed.

3.4 Tools and Technologies Used

- **Programming language:** TypeScript on Node.js 20 or later (tests recorded on Node 20, 22 and 24).
- **Libraries and frameworks:** `ioredis` for Redis; Node's built-in `node:crypto`; WebCrypto, IndexedDB and `BroadcastChannel` in the browser; Express, Fastify, Hono and Koa for adapters and examples; `jose` in the SecureShop JWT baseline.
- **Build and test:** `tsup` for dual ESM and CommonJS builds, Vitest for tests, `fake-indexeddb` for client key-store tests, Playwright 1.64.1 for the Chromium, Firefox and WebKit regressions.
- **Infrastructure:** Redis 7 (Docker images `redis:7-alpine`), npm workspaces, GitHub Actions.
- **Hardware and operating systems: the fresh re-run of 21 October 2026 used a Dell Latitude E5440 (Intel Core i5-4300U, 2 cores / 4 threads, 16 GB RAM) running Windows 10 Home (build 19045) with Redis 7.4.11 in Docker Desktop (WSL2); CI runs on Ubuntu.**

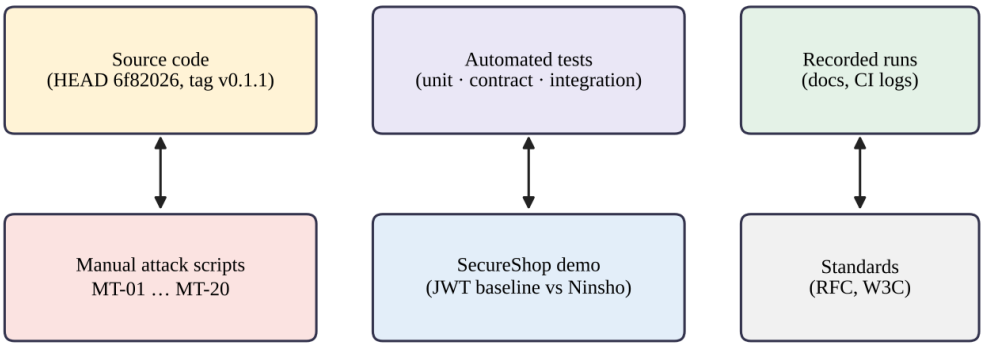


Figure 3.6: Evidence sources used in this report

3.5 System Context and Deployment

Figure 3.7 places Ninsho in its environment. Ninsho is not a separate service: it is a library loaded into the host application's Node.js process, so the application and Ninsho share one trust domain. The browser, the network and any authenticator are outside it. Redis is a separate process whose availability and durability Ninsho depends on but cannot control, and the user database belongs to the application.

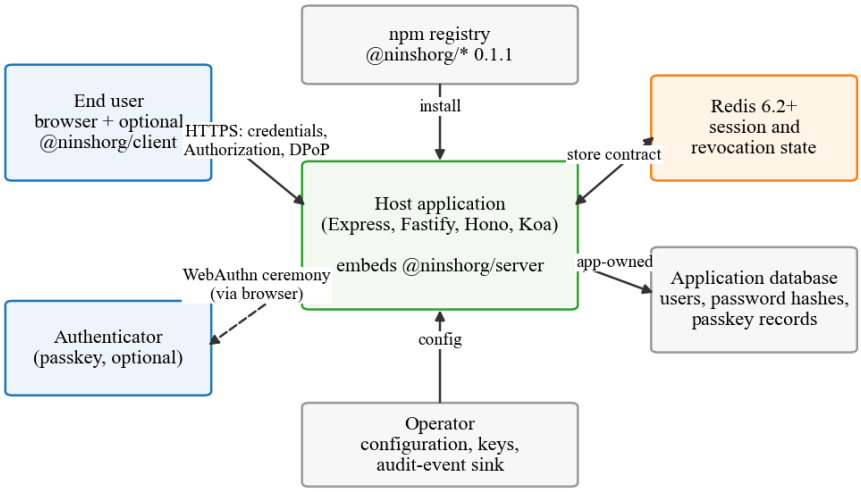


Figure 3.7: System context of a deployment that uses Ninsho

The actors are therefore the end user and browser (untrusted), the optional browser client (running in the browser and so also untrusted), the host application and its operator (trusted, same process), Redis (trusted for integrity, not for availability), any authenticator (verified, not trusted) and the npm registry as a supply-chain dependency. This distinction between what Ninsho controls and what it assumes is the basis of the threat model in Section 3.6.

Figure 3.8 shows the package boundaries. The dependency direction is deliberate: `@ninshorg/core` depends on nothing, the server and WebAuthn packages depend only on core (and the server on ioredis for its Redis store), and the browser client depends on nothing and talks to the server only over HTTP. These boundaries are enforced by the bundle-purity job in continuous integration, which fails the build if a third-party runtime dependency appears in core, client or webauthn, or if a framework is imported by an adapter.

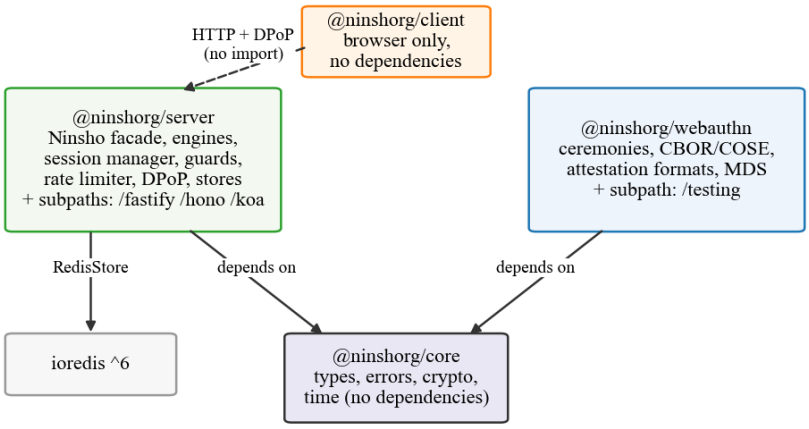


Figure 3.8: Package architecture and runtime dependencies (release 0.1.1)

A production deployment normally runs several instances of the application behind a reverse proxy, as in Figure 3.9. Because every instance reads and writes the same Redis keys, a logout handled by instance A takes effect on instance B at its next lookup, without any message between the instances. The proxy matters for two features: DPoP proofs are bound to a URI that Ninsho reconstructs from the Host header unless the application supplies a builder, and the per-address rate limit depends on how many proxies are trusted to append X-Forwarded-For.

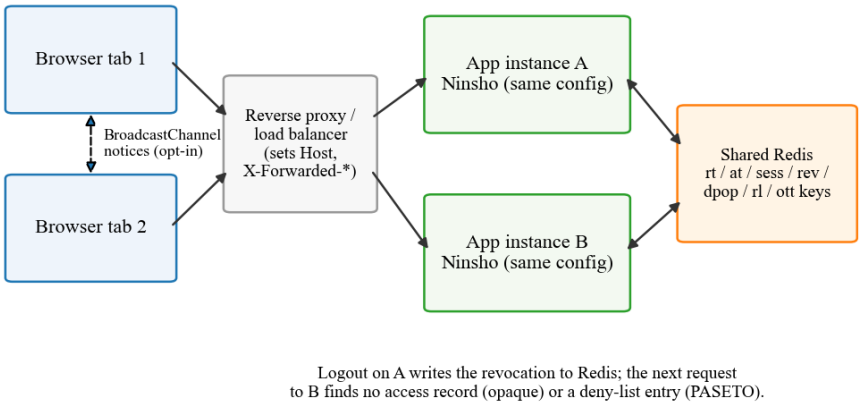


Figure 3.9: Multi-instance deployment with a shared Redis store and synchronized browser tabs

3.6 Threat Model

The threat model below follows the repository's SECURITY.md and the review carried out for this report. It identifies what is protected, who may attack it, where untrusted input crosses into trusted code, and which threats are explicitly out of scope.

3.6.1 Assets

Table 3.4 lists the assets that Ninsho protects and where each one lives.

Table 3.4: Assets protected by Ninsho

Asset	Why it matters	Where it lives
Access tokens	Grant access to protected routes	Client memory; store holds only SHA-256 hashes (opaque) or nothing (PASETO)
Refresh tokens and their families	Renew sessions for up to seven days	HTTP-only cookie at the client; hashes, tombstones and grace mappings in the store
Revocation state	Makes logout and theft response effective	Store keys sess:*:dead*, *rtx:*, rev: *and the absence of at:* records
DPoP private keys	Make bound tokens useless without the key	Browser IndexedDB, non-extractable
PASETO signing keys	Allow forging any token if leaked	Server configuration (KeyRing)
WebAuthn challenges and credential bindings	Prevent ceremony replay and cross-account sign-in	Challenge store; application's credential records
One-time tokens	Grant password reset or email verification	Email link; store holds only the hash
Rate-limit counters	Slow credential guessing	Store keys rl:*
Audit trail	Allows detection and investigation	Application-supplied audit sink

3.6.2 Adversaries and trust boundaries

Figure 3.10 shows the trust boundaries and the adversaries considered. Seven boundaries are identified. TB1 is the HTTP entry to verify(), where the Authorization and DPoP headers, the Host header and the request target are attacker-controlled. TB2 is the rate limiter, which sees the socket address, forwarding headers and the submitted account identifier. TB3 covers the refresh and logout routes, where timing and concurrency are under the attacker's control. TB4 is the connection to Redis, whose availability, persistence and eviction policy affect every guarantee. TB5 is WebAuthn verification, TB6 the boundary between page script and the client library, and TB7 the configuration chosen by the integrating developer. Table 3.5 gives each adversary's assumed capability, the controls that address it and what is not defended.

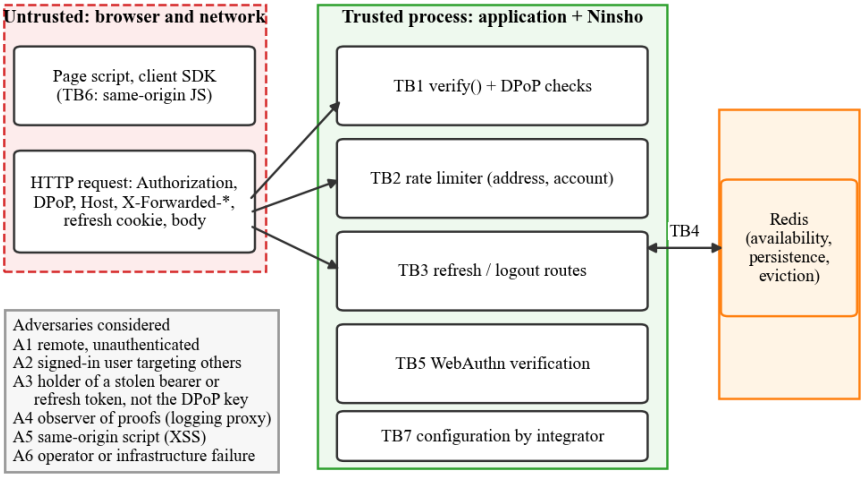


Figure 3.10: Trust boundaries and adversaries considered in the threat model

Table 3.5: Adversaries, capabilities and primary controls

Adversary	Capability assumed	Primary controls	Not defended
A1 Remote, unauthenticated	Send arbitrary HTTP requests	Token verification, rate limiting, generic error messages	Volumetric denial of service
A2 Signed-in user targeting others	Valid session; can change identifiers in requests	requireOwner, requireTenant, role and scope guards	Routes where the application omits the guard
A3 Thief of a bearer or refresh token	Holds a copied token, not the DPoP key	Short access lifetime, rotation with reuse detection, DPoP binding	Use of an unbound access token before it expires or is revoked
A4 Observer of proofs	Reads proofs from a log or proxy	Single-use proof identifiers, URI and method binding, 60-second age limit	Server-nonce freshness (not implemented)
A5 Same-origin script (XSS)	Runs code in the application's origin	Access token kept in memory; key non-extractable	Use of the key for signing while the script runs

Adversary	Capability assumed	Primary controls	Not defended
A6 Operator or infrastructure failure	Redis outage, restart without data, rollback	Fail-closed default; documented behaviour per strategy	PASETO revocations lost with the store

3.6.3 Security objectives and out-of-scope threats

From the assets and adversaries follow the security objectives that the design must meet:

- **O1** A credential that has been revoked is refused at the next request on every instance that shares the store.
- **O2** A refresh token presented after its replacement has been issued, outside the grace window, ends the whole session and raises an alarm.
- **O3** A token bound with DPoP is refused without a fresh, unreplayed proof from the bound key.
- **O4** When the state needed to decide is unavailable, the default response is refusal, not acceptance.
- **O5** Authorization guards refuse when they cannot establish the answer.
- **O6** Nothing that the store holds is a usable credential, apart from the short-lived grace mapping that is documented.

The following threats are out of scope and are stated so that the objectives are not over-read: compromise of the application server or of Redis itself; malicious script in the application's origin using the client's key while it runs; phishing of passwords before Ninsho is involved; denial of service by traffic volume; side channels in the Node.js runtime; and misconfiguration by the integrator beyond the cases that the configuration validator refuses.

3.7 Detailed Lifecycle Design

This section describes each lifecycle operation at the level of individual store operations. The sequence diagrams were drawn from the source files named in each subsection, and the pseudocode summarizes the same code. The pseudocode is explanatory and omits logging, type checks and error wrapping; it is not executable production code.

3.7.1 Session creation and access-token verification

Figure 3.11 shows sign-in and a subsequent protected request with the default opaque strategy (packages/server/src/session/manager.ts, engine/opaque.ts and

http/middleware.ts). After the application has verified the user's credentials, `createSession()` writes the refresh record, the session metadata and the user's session index, and the engine writes the access record and its indexes. Before returning, the manager checks whether a revocation marker for the new session already exists; this closes a race in which a concurrent logout-all could enumerate the user's sessions before the new token was indexed.

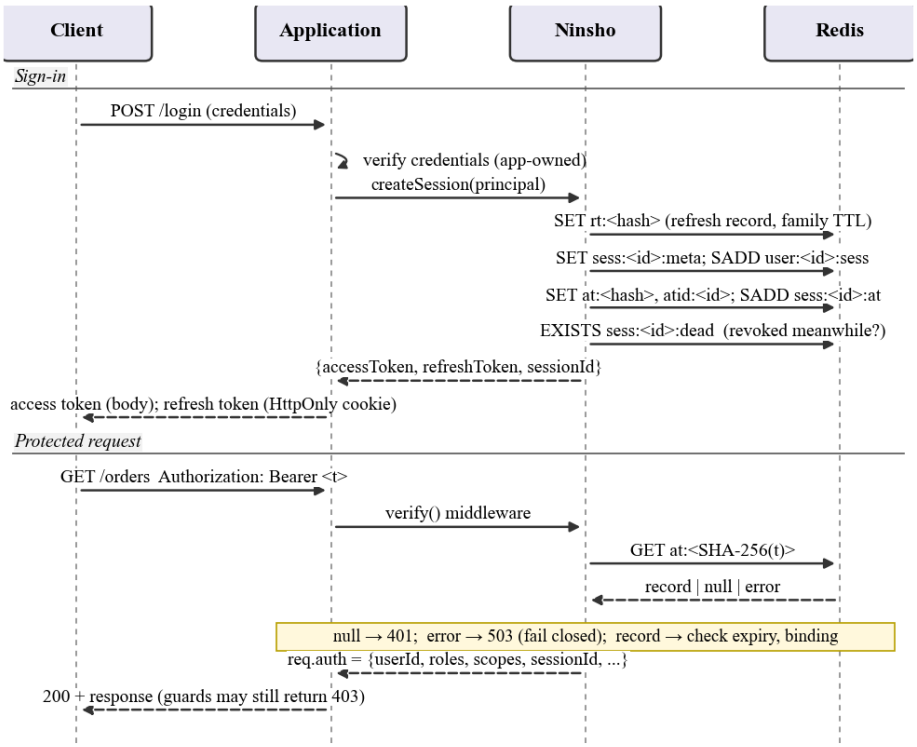


Figure 3.11: Sequence of session creation and verification of an opaque access token

```
verify(request):                                    // explanatory pseudocode
  token := single Authorization header (Bearer or DPoP scheme)
  if several Authorization headers in rawHeaders: reject 401
  if binding = dpop: jkt := establishProofOfPossession(request, token)
  try:
    record := store.get('at:' + SHA256(token))      // the only store read
  on store error:
    if onStoreError = closed or engine is opaque: emit store.unavailable;
reject 503
    else: verify PASETO signature only; emit store.unavailable (admitted)
  if record is null: reject 401 TOKEN_INVALID     // never existed, expired or
revoked
  if record.expiresAt + tolerance < now: revoke record; reject 401
TOKEN_EXPIRED
  if record.confirmationKey and record.confirmationKey != jkt: reject 401
  request.auth := {userId, roles, scopes, tenant, sessionId, authenticatedAt,
...}
```

The single read serves both authentication and revocation: a revoked opaque token has no record, so the same lookup that identifies the caller enforces revocation. With the PASETO strategy the lookup is replaced by a signature check followed by an `EXISTS rev:<jti>`

deny-list check, so a PASETO token is accepted without the store only when the operator has selected fail-open.

3.7.2 Refresh-token rotation and reuse detection

Figure 3.4 summarised the states a refresh token passes through; Figure 3.12 shows the most delicate operation: two tabs presenting the same refresh token simultaneously, followed by a replay after the grace window. The design rests on three ordering rules taken from manager.ts. First, a DPoP-bound refresh token's binding is checked by *peeking* at the record before it is consumed, so that a thief without the key cannot destroy the victim's session by presenting the token. Second, consumption uses the store's atomic `take` (Redis GETDEL), so exactly one caller rotates. Third, the winner writes the replacement and the grace mapping *before* the tombstone, so that any caller who sees a tombstone also sees the grace mapping and adopts the replacement instead of reporting reuse.

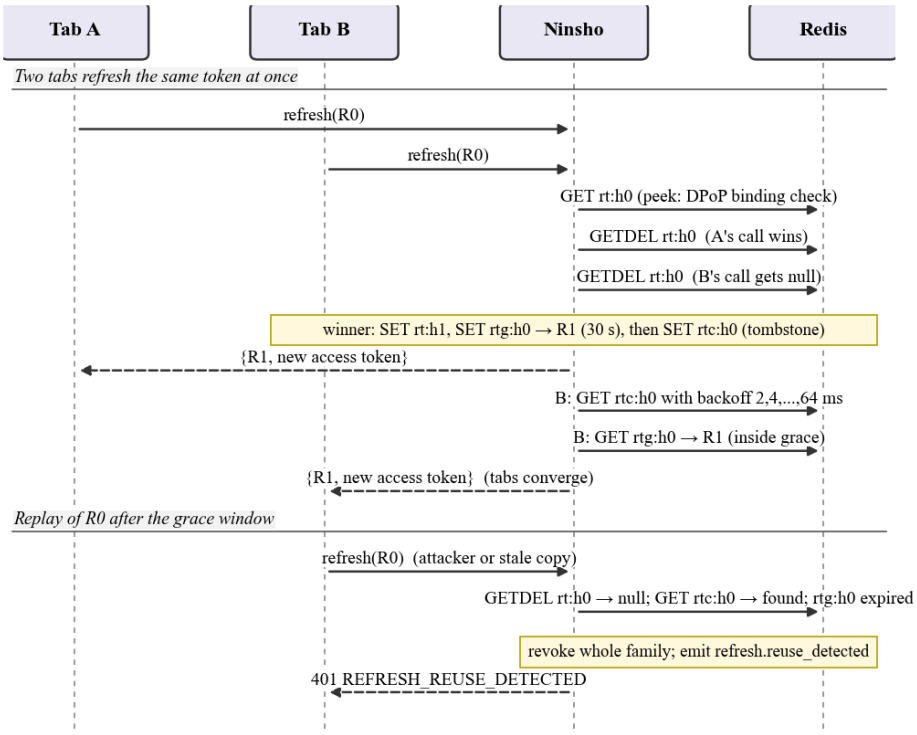


Figure 3.12: Sequence of refresh-token rotation with a parallel tab and a later replay

```
refresh(rawToken, options):                          // explanatory pseudocode
  h := SHA256(rawToken)
  peek := store.get('rt:' + h)
  if peek is bound and options.key != peek.key: refuse (nothing consumed)
  live := store.take('rt:' + h)                      // atomic GETDEL
  if live: return rotate(h, live)
  tomb := get 'rtc:' + h, retrying with backoff 2,4,8,16,32,64 ms
          (skipped if grace is 0; answered at once if 'rtx:' + h exists)
  if tomb is null: refuse REFRESH_INVALID            // unknown: nothing revoked
  if tomb is bound and options.key != tomb.key: refuse
  grace := store.get('rtg:' + h)
  if grace: return same replacement + new access token  // parallel tab
  revokeFamily(tomb.sessionId, 'reuse_detected'); emit refresh.reuse_detected
  refuse REFRESH_REUSE_DETECTED

rotate(h, record):
  check binding again; refuse if session marker 'sess:<id>:dead' exists
  refuse if the token or the family has expired (family revoked silently)
  write new token (generation + 1, expiry = min(now + TTL, family expiry))
  write grace 'rtg:' + h -> new token (30 s); then tombstone 'rtc:' + h
  re-check the session marker; issue the access token; re-check again
  return {accessToken, refreshToken}
```

Rotation carries the original authentication time forward unchanged, so a session that is refreshed for days never satisfies `requireFreshAuth()`; only a new sign-in does. The bounded backoff exists because the tombstone is written a few store round trips after the consume; measured against local Redis, the repository's comments record that a two-caller race takes about 14 ms at the median, and the backoff totals 126 ms so that a loser does not report a spurious failure.

3.7.3 Logout and revocation

Figure 3.13 shows revocation of one session. The first write is a revocation marker for the session, and every path that issues a credential re-checks that marker after writing. The consequence is the property stated at the bottom of the figure: either the revocation enumerates the newly issued token, or the issuing path observes the marker and withdraws what it issued. Revocation of all of a user's sessions works from one snapshot of the user's session index, removes only the sessions that were revoked successfully, and raises `RevocationIncompleteError` if any failed so that the caller can retry. The individual steps are those of Section 3.3.3: marker, refresh-family records, access tokens through the engine, and finally the user's index.

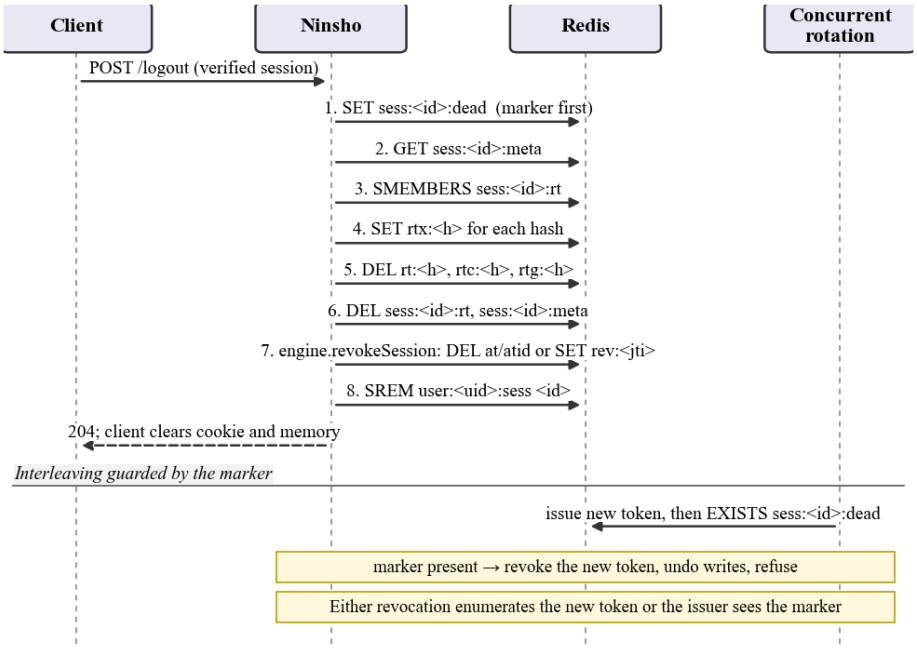


Figure 3.13: Sequence of session revocation and the marker that guards a concurrent rotation

3.7.4 DPoP proof validation and replay protection

Figure 3.5 shows the order of the checks on a DPoP-bound request (http/dpop-middleware.ts, dpop/proof.ts, dpop/jwk.ts and dpop/replay.ts). The middleware first refuses requests with more than one Authorization or DPoP header, consulting the raw header list because Node.js silently keeps only the first Authorization header. The proof's type must be `dpop+jwt`, its algorithm must be ES256 or EdDSA and must match the embedded public key, and no private key member may be present. The signature is verified before any claim is read. The method, the URI reconstructed from the server's view of the request, the issue time and, when a token is presented, the hash of that token are then compared. Finally the proof identifier is claimed with an atomic set-if-absent keyed by the key thumbprint, retained for the proof age plus twice the clock tolerance plus one second (71 seconds with the defaults), so a captured proof works once.

3.7.5 Authorization guards

The guards run after verify() and read the request context it populated; calling one without verify() throws instead of evaluating an empty identity. requireRole and requireScope pass when the caller has any listed value, and requireAllRoles when it has all of them; an empty list is rejected when the guard is constructed. requireOwner passes only when its selector returns exactly the caller's user identifier, and refuses when the selector throws or returns nothing, an empty string, an array (as an Express 5 repeatable parameter can) or another

identifier. requireTenant additionally refuses a token with no tenant, and requireFreshAuth refuses when the session's original authentication is older than the limit or cannot be read. Each guard emits an `authz.denied` audit event and answers 403 with a deliberately uninformative message, so that a refusal does not confirm that another user's resource exists.

3.7.6 Rate limiting and single-use tokens

The rate limiter uses a sliding-window counter: two integers per bucket, the current and previous fixed windows, combined as previous × (1 − elapsed fraction) + current. This avoids the double burst that a fixed window permits at its boundary while keeping memory bounded, unlike a log of every request. Every request is counted in two buckets, one keyed by client address and one by account identifier, and both are always consumed so that an attacker cannot keep one counter low by tripping the other. The `trustProxy` option has no default and must be stated, because keying on the wrong address either places every user behind a proxy in one bucket or lets a client choose its own address.

Single-use tokens for password reset, email verification and magic links are stored by hash under a key that includes their purpose, so a reset token cannot be redeemed at a weaker endpoint. They are consumed with the atomic `take`, so two simultaneous clicks cannot both succeed, and issuing a new token for the same subject and purpose increments a generation counter that invalidates the older token. The default lifetime is 900 seconds.

3.7.7 WebAuthn verification responsibilities

Figure 3.14 separates the responsibilities in a passkey sign-in. The WebAuthn package issues and consumes single-use challenges and verifies the ceremony: the client-data type, the challenge, the origin, the hash of the relying-party identifier, the user-presence and user-verification flags, the signature, the sign counter and, for registration, the attestation statement under the configured policy. Everything else belongs to the application: it stores the credential identifier, public key, counter and owner, looks the credential up when an assertion arrives, supplies the owner to the verifier, updates the counter and finally calls `createSession()` with the returned principal.

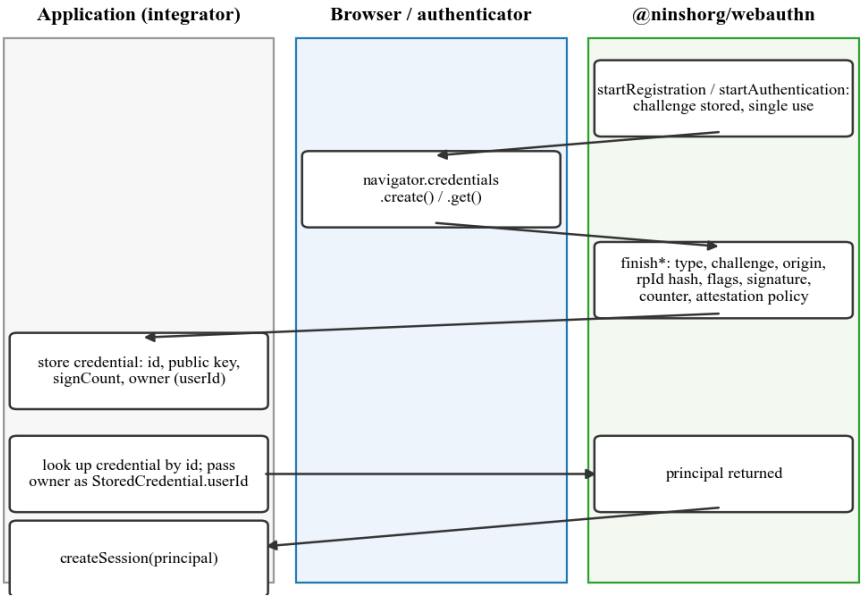


Figure 3.14: Division of responsibilities in a WebAuthn registration and sign-in

The importance of the owner field is discussed in Section 4.5: in the published release 0.1.1 the stored credential's owner is optional, and an application that omitted it could have accepted a sign-in for the wrong account. A later working-tree change makes the server refuse such a credential; that change is not part of the published release.

3.8 Implementation Details

3.8.1 Public interface of the server facade

Most applications use only the `Ninsho` class, constructed with one configuration object (packages/server/src/ninsho.ts). It exposes the session operations createSession(), refresh(), revokeSession(), revokeAllForUser() and listSessions(); the verify() middleware and the six guards; confirmProofOfPossession() and setRequestUrlBuilder() for DPoP; rateLimit() and oneTimeTokens; and errorHandler(), health() and close() for operations.

3.8.2 Configuration and secure defaults

Only `store` is required. The configuration is validated synchronously in the constructor, so a misconfigured deployment fails at start-up rather than at its first request. Table 3.6 lists the options, their defaults and their security effect. Several unsafe combinations are refused outright: fail-open with opaque tokens (the store holds the identity, so there is nothing to fall back on), a refresh lifetime not longer than the access lifetime, a grace window as long as the refresh lifetime, a clock tolerance longer than the access lifetime, an access lifetime

above 86,400 seconds, and PASETO-only settings with the opaque strategy. Other accepted but weakening choices emit a `config.insecure` audit event at start-up.

Table 3.6: Configuration options of the Ninsho constructor

Option	Default	Purpose	Security effect of changing it
store	(required)	Where all state lives	MemoryStore refuses NODE_ENV=production
strategy	opaque	Access-token format	paseto enables local verification; revocation then depends on a deny-list
accessTokenTtl	300 s	Access-token lifetime	Above 3,600 s warns; above 86,400 s refused
refreshTokenTtl	604,800 s	Refresh family lifetime	Must exceed accessTokenTtl
refreshGraceSeconds	30 s	Tolerance for parallel tabs	Above 120 s warns: replays inside the window are not detected; 0 disables
onStoreError	closed	Behaviour during a store outage	open (PASETO only) admits revoked tokens during the outage; warns
binding	none	Bearer or DPoP semantics	dpop requires a client key and a proof on every request
dpopProofMaxAgeSeconds	60 s	How old a proof may be	Above 300 s warns; only valid with binding dpop
clockToleranceSeconds	5 s	Allowed clock skew	Must not exceed accessTokenTtl
issuer, audience, keys	(none)	PASETO identity and keys	Required for paseto; refused for opaque
audit	JSON lines to stdout	Where security events go	Must implement emit()
oneTimeTokens	900 s, 32 bytes	Single-use token settings	TTL capped at seven days; at least 16 bytes

3.8.3 Store contract and key schema

Every piece of state passes through the `NinshoStore` interface (packages/server/src/store/types.ts): get, set, setIfAbsent, take, increment, delete, exists, sAdd, sRemove, sMembers, ping and close. Two operations carry the correctness of the whole library and must be atomic: `take`, which reads and deletes in one step and is implemented with Redis GETDEL, and `setIfAbsent`, implemented with a single SET NX EX command. The Redis implementation disables the client's offline queue, so commands issued during an outage fail at once instead of resolving after the HTTP request has already timed out. The excerpt below is from packages/server/src/store/redis.ts.

```
async take(key: string): Promise<string | null> {
  // GETDEL is atomic; a GET followed by DEL is not, and would let two
  // concurrent callers both observe the value before either deleted it.
  return this.#withClient((c) => c.getdel(key));
}
```

All keys are generated in one module, packages/server/src/keys.ts, under the versioned prefix `ninsho:v1:` so that a future schema change can coexist with the current one. Table 3.7 lists them; a reviewer can predict from this table every key a deployment writes.

Table 3.7: Store key schema (prefix ninsho:v1:)

Key pattern	Holds	Lifetime
at:<hash>	Opaque access record (principal, session, expiry, binding)	Access TTL
atid:<tokenId>	Reverse index from token id to hash, for logout	Access TTL
sess:<id>:at	Set of the session's access tokens	Opaque: access TTL + 60 s; PASETO: + tolerance as well (F-01 fix)
rt:<hash>	Live refresh record	Until token expiry
rtc:<hash>	Tombstone of a consumed refresh token	Until family expiry
rtg:<hash>	Grace mapping to the replacement token (the one raw token stored)	Grace window (30 s)
rtx:<hash>	Marker that the token's family was revoked	Refresh TTL
sess:<id>:rt / :meta / :dead	Family index, metadata, revocation marker	Family lifetime / refresh TTL
user:<id>:sess	User's session index	Reset to the newest session's family lifetime
rev:<jti>	PASETO deny-list entry	Access TTL + tolerance + 1 s
dpop:<jkt>:<jti>	Claimed DPoP proof identifier	Proof age + 2 × tolerance + 1 s
ott:<purpose>:<hash>, ott:<purpose>:gen:<subject>	One-time token and its generation counter	Token TTL; seven days + one hour
rl:<bucket>:<window>	Rate-limit counter	Three windows

The lifetime of `sess:<id>:at` for PASETO tokens is the subject of finding F-01 in Section 4.5: in release 0.1.1 it was the access lifetime plus 60 seconds, which is shorter than the token's acceptance window when the clock tolerance exceeds 60 seconds. The working-tree fix extends it by the tolerance; the table shows the corrected value, which is not yet released.

3.8.4 Error model and audit events

Every error extends `NinshoError` and carries two separate texts: a fixed, uninformative `message`, which is the only text sent to a client, and a `detail` for server logs, which the response helper never reads. Authentication failures deliberately share messages, so that a client cannot distinguish an expired token from a revoked or never-issued one. Table 3.8 lists the error classes. Security-relevant events are emitted to the audit sink: session.created,

session.refreshed, session.revoked and session.revoked_all for the lifecycle; refresh.reuse_detected when a rotated refresh token is replayed; token.rejected when a DPoP proof is replayed; authz.denied and ratelimit.exceeded for refusals; onetime.issued and onetime.consumed for single-use tokens; store.unavailable whenever the store could not be reached, whether the request was refused or admitted; and config.insecure for weakening settings accepted at start-up. Two further event types declared in the type definitions, token.revoked and auth.failed, are not emitted by the server package at commit 6f82026.

Table 3.8: Error classes, codes and HTTP statuses (packages/core/src/errors.ts)

Class	Code	Status	Client message
TokenMissingError	TOKEN_MISSING	401	Authentication required
TokenInvalidError	TOKEN_INVALID	401	Invalid authentication credentials
TokenExpiredError	TOKEN_EXPIRED	401	Authentication credentials have expired
TokenRevokedError	TOKEN_REVOKED	401	Authentication credentials are no longer valid
RefreshInvalidError	REFRESH_INVALID	401	Session could not be renewed
RefreshReuseError	REFRESH_REUSE_DETECTED	401	Session could not be renewed
ForbiddenError	FORBIDDEN	403	Insufficient permissions
RateLimitError	RATE_LIMIT_EXCEEDED	429	Too many requests
StoreUnavailableError	SERVICE_UNAVAILABLE	503	Service temporarily unavailable
ConfigurationError, KeyError	CONFIGURATION_ERROR, KEY_ERROR	500	(start-up errors)
RevocationIncompleteError	REVOCATION_INCOMPLETE	500	Not all sessions could be revoked

3.8.5 Integration responsibilities

Because Ninsho is a library, a correct deployment depends on the application performing its share. Table 3.9 states the division explicitly; several of the limitations in Chapter 4 arise where the application side of this table is not met.

Table 3.9: Responsibilities of Ninsho and of the integrating application

Concern	Ninsho	Integrating application
User authentication	Not performed	Verify passwords or passkey assertions; call createSession() only on success
Refresh-token transport	Issues and rotates the token	Store it in an HTTP-only, Secure cookie; clear it on refusal
Authorization	Provides guards	Mount verify() and the right guard on every protected route; supply a trustworthy owner selector

Concern	Ninsho	Integrating application
Rate limiting	Provides middleware	State trustProxy correctly for the deployment; choose limits
DPoP	Verifies proofs and binding	Use the client library or equivalent; configure the URI builder behind untrusted proxies
WebAuthn	Verifies ceremonies	Store credentials with their owner; update counters; enforce enrolment policy
Account events	Revokes sessions on request	Call revokeAllForUser() after password change or suspected compromise
Operations	Emits audit events; fails closed	Run Redis 6.2+ with suitable persistence; alert on store.unavailable and refresh.reuse_detected

The Express example application shows the intended pattern. The following excerpt is copied from examples/express-api/src/app.ts: logout is an authenticated route that revokes the verified session, and a per-user resource route mounts the ownership guard after verification.

```
app.post('/auth/logout', auth.verify(), route(async (req, res) => {
  await auth.revokeSession(getAuth(req).sessionId, 'logout');
  clearRefreshCookie(res);
  res.status(204).end();
}));

app.get(
  '/users/:id/orders',
  auth.verify(),
  auth.requireOwner((req) => req.params?.['id']),
  (_req, res) => { res.json({ orders: [] }); },
);
```

3.8.6 Security controls and their limits

Each control described in this chapter has a corresponding limitation: hashing at rest excepts the 30-second grace mapping; reuse detection cannot distinguish a replay inside the grace window; DPoP cannot stop same-origin script from signing and has no server nonce; the replay guard and rate limits are suspended under fail-open; and guards protect only the routes on which they are mounted. Section 4.5 evaluates these controls against the findings of the review.

3.9 SecureShop Consumer Showcase

SecureShop (demo/viva-showcase) is a small shop application written for the viva. It installs the published @ninshorg/server and @ninshorg/client packages from npm, as an external consumer would, and implements the same pages three times: with a stateless HS256 JWT stored in localStorage (Comparison A), with Ninsho opaque tokens on two application instances that share Redis (Comparison B), and with Ninsho DPoP binding, an in-memory access token, a non-extractable key and cross-tab synchronization (Comparison

C). A Security Experiments Lab page issues replay and forgery requests so that an examiner can watch the responses. Table 3.10 states what each comparison demonstrates and what it does not establish.

Table 3.10: What the SecureShop comparisons demonstrate and what they do not establish

Comparison	Demonstrates	Does not establish
A: stateless JWT	A token copied before client logout is still accepted afterwards (HTTP 200)	That JWT systems cannot revoke; this baseline deliberately has no deny-list
B: Ninsho opaque on two instances	Logout through instance A is refused on both A and B (HTTP 401)	Behaviour under real network partitions or Redis failover
C: Ninsho with DPoP	Token-only, wrong-key and replayed-proof requests are refused (HTTP 401); peer tabs are notified	Protection against script running in the page
All	Integration of the published packages by a separate application	Production deployment, real users, or adoption

The demonstration accounts and data are seeded and illustrative, and their credentials must never be used in a real system. SecureShop is evidence that the published packages can be integrated and that the documented outcomes occur under its configuration; it is not evidence of production readiness.

Chapter 4

Results and Discussion

This chapter reports only what the repository, public CI records and the project's own documents support. Each result is labelled with who produced it and whether it was independently re-executed while this report was prepared. Passing tests are evidence that specified behaviours held under the tested conditions. They do not demonstrate that all attacks are prevented.

4.1 Experimental Setup

Table 4.1 classifies the evidence used. "Project-recorded" means the result is documented in the repository as a run by the project's authors. "CI-confirmed" means the result was read from GitHub's public API for the project's Actions run on the commit examined. "Re-run" means the result was reproduced on 21 October 2026 on the machine in Table 4.2; its logs, probes and benchmark files are in the repository under research/validation. Figure 3.6 summarises the sources of this evidence.

Table 4.1: Evidence categories and their status

Evidence	Source	Status
Full test suite with real Redis	Repository documents (review remediation, cross-tab guide, browser guide)	Project-recorded; not re-run for this report
CI on commit 6f82026	GitHub Actions run 38607481028 (push to main, 14 Oct 2026), all 8 jobs succeeded	CI-confirmed (read on 20 Oct 2026)
Browser regressions (3 engines)	Local results in the browser guide; CI matrix of three jobs	Project-recorded and CI-confirmed
Manual attack procedures MT-01 to MT-20	`manualtest/TEST-RESULTS.md` and evidence files, written against release 0.1.0	Project-recorded; not re-run on 0.1.1
Redis persistence scenarios	`redis-persistence.test.ts` and operations guide	Project-recorded; CI execution of the Docker-managed suite not established
SecureShop outcomes	SecureShop README and test file	Re-run 21 Oct 2026: 3/3 passed; 4/4 after the B-12 fix (logs in research/validation/logs)
Benchmarks	Performance documents for release 0.1.0	Historical; not for the current commit
Fresh re-run by the authors	21 Oct 2026; commit 6f82026 plus local fixes made during the final review; Node 22.13.1, npm 11.0.0, Redis 7.4.11; logs in research/validation/logs	Re-run: build, typecheck, 2,152 tests, 6 browser runs, 7 Redis persistence scenarios, SecureShop 4/4

Table 4.2: Experimental environments recorded in the repository

Item	Recorded value
Commit examined	6f82026 (feat(demo): add SecureShop viva showcase application); packages published as 0.1.1 from commit f784566 with identical package source
Local runs (project-recorded)	Windows; Node 22.13.1 (browser and cross-tab runs) and 24.19.0 (viva rehearsal); npm 11.0.0; dedicated Redis 7.4.8 and 7.4.11 in Docker; Playwright 1.64.1
CI	GitHub Actions: Ubuntu runners; Verify on Node 20 and 22 with a `redis:7-alpine` service; browser jobs on Ubuntu 24.04 with Node 22; lockfile, dependency audit and bundle-purity jobs
Local hardware	Fresh re-run: Dell Latitude E5440, Intel Core i5-4300U (2 cores / 4 threads), 16 GB RAM, Windows 10 Home 10.0.19045, Docker Desktop 4.38.0

4.2 Results

4.2.1 Automated test suite and continuous integration

The project recorded a full run (build, type-check, tests) against a dedicated Redis instance with no skipped tests. The browser guide and the cross-tab guide record 2,142 workspace tests in 53 files plus one HTTP demonstration test; the review-remediation document records 2,116 tests with no skips on Node 20, 22 and 24.19.0. The two figures describe different working-tree states of the project and neither is a count for the final commit, so this report does not present either as the count for commit 6f82026. Figure 4.1 shows the per-package breakdown recorded with the 2,142 figure.

A fresh run on 21 October 2026 against a dedicated Redis 7.4.11 instance, on commit 6f82026, passed 2,150 tests with none skipped (client 117, core 97, server 1,116, webauthn 623, Express example 101, playground 95, HTTP demonstration 1); all seven Docker-managed Redis persistence scenarios executed. After adding one regression test for defect F-01 the count is 2,151. In the first post-fix run two Docker persistence tests failed while starting a container (a container-name conflict while Docker was still removing the previous one); the server package was re-run immediately and all 1,117 of its tests passed. This is reported as an intermittent test-harness failure, not hidden. With the further regression tests added during the final review, the full suite passed 2,152 tests with none skipped.

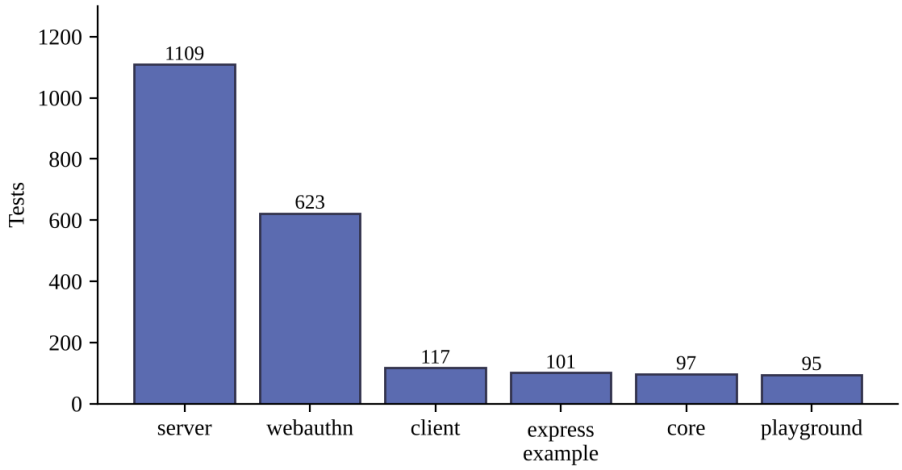


Figure 4.1: Tests per workspace in the recorded run (project-recorded)

On the commit examined, all eight jobs of the GitHub Actions run succeeded: Verify on Node 20 and Node 22 (with a Redis service), Chromium, Firefox and WebKit browser jobs, the lockfile-consistency job, the dependency audit of production dependencies, and the bundle-purity job, which also installs the packed tarballs as a consumer would and loads them as ESM and CommonJS. The single skipped step was the benchmark smoke run on Node 20, which is skipped by design. One caveat applies: the CI step that checks that the Redis persistence suite ran searches the output for the suite name, and the placeholder printed when Docker is unavailable has the same name, so that step alone cannot show the Docker-managed persistence tests executed on the runner.

4.2.2 Security behaviours and their test evidence

Table 4.3 maps the principal security behaviours to the test files that the repository's documentation names for them. The mapping was checked against the file list at the commit examined; passing status derives from the successful CI run above and from the project-recorded runs.

Table 4.3: Security behaviours and the test files that evidence them

Behaviour	Primary test evidence (file in repository)	Status
Refresh rotation, single winner, reuse detection and grace window	`session.test.ts`, `store-invariants.test.ts`	Passing (CI)
Revocation racing initial issuance; sender-binding checked before consumed-token reuse handling	`review-regressions.test.ts`	Passing (CI)
DPoP: stolen token useless without key; refresh tokens bound	`dpop-integration.test.ts`	Passing (CI)
DPoP proof parsing and thumbprint against the RFC 7638 vector	`dpop-proof.test.ts`	Passing (CI)
DPoP replay rejected; method and URI mismatch rejected	`dpop-request.test.ts`, `dpop-integration.test.ts`	Passing (CI)

Behaviour	Primary test evidence (file in repository)	Status
Ownership, role, scope, tenant and fresh-authentication guards	`middleware.test.ts`, `fresh-auth.test.ts`	Passing (CI)
PASETO v4.public official vectors and tamper resistance	`paseto-v4.test.ts`, `paseto-independent-audit.test.ts`	Passing (CI)
Store contract holds for MemoryStore and RedisStore; fail-closed against an unreachable Redis	`store.contract.test.ts`, `store-invariants.test.ts`	Passing (CI, Redis job)
Rate limiting without boundary double-burst; fail-closed on store error	`ratelimit.test.ts`	Passing (CI)
Browser key store, lifecycle races and cross-tab sync	`keys.test.ts`, `storage.test.ts`, `lifecycle.test.ts`, `session-sync.test.ts`	Passing (CI)
WebAuthn ceremonies, attestation formats, CBOR and COSE parsing	`ceremony.test.ts`, `attestation.test.ts`, `cbor.test.ts`, `cose.test.ts`	Passing (CI)

The repository also contains an in-repository cryptographic review of the PASETO implementation. It adds an independent reference implementation of the pre-authentication encoding and 44 tests, including exhaustive single-byte mutations of the signature. This was carried out within the project.

4.2.3 Manual attack procedures

Twenty manual procedures, with five follow-up retests, were recorded against the example applications. They give 25 recorded entries: 23 passed and 2 were marked not applicable because no resource endpoint of the required kind existed in the shipped applications. By Equation (3.3), the conformance over applicable entries was 23 of 23. These procedures were recorded against release 0.1.0 and have not been re-run on 0.1.1. Table 4.4 groups them.

Table 4.4: Manual attack procedures grouped by theme (project-recorded, release 0.1.0)

Theme	Entries	Recorded outcome
Token tampering and signature, footer, issuer and audience attacks	MT-01 to MT-03, MT-01B, MT-19, MT-19B	Rejected with 401
Replay: revoked token, expired token	MT-04, MT-05, MT-05B, MT-06	401 after logout and after expiry
Refresh replay and rotation abuse	MT-07, MT-08	Reuse after the grace window rejected and family revoked; concurrent refresh converged on one replacement token
Authorization: cross-user, role, scope, tenant	MT-09 to MT-12	403 for other users' resources and for missing role; MT-11B and MT-12B not applicable
Rate limiting	MT-13	Fifth failed attempt allowed; sixth returned 429 with a retry hint
DPoP: missing proof, method, URI and replay	MT-14 to MT-17	Rejected in each case
Step-up authentication and malformed input	MT-18, MT-20	Stale authentication refused; oversized body 413; malformed cookie handled without a crash

4.2.4 Browser evidence

Two regression scripts run unchanged scenarios on Chromium, Firefox and WebKit through Playwright. The key-store script gates two pages' initial IndexedDB loads, creates keys concurrently and checks that both pages receive the same non-extractable key which survives reload. The session-sync script checks that a peer logout invalidates state, that a delayed refresh cannot restore it, that a delayed sign-in cannot override a peer identity change, and that the shared key survives logout. Local results recorded by the project are in Table 4.5; the three browser jobs in the GitHub Actions run on Ubuntu also succeeded. The local and CI results are distinct measurements and are reported separately.

Table 4.5: Browser regression results (project-recorded local run on Windows, Node 22.13.1, Playwright 1.64.1)

Engine	Version reported	Key-store	Session-sync	CI job on 6f82026
Chromium	156.0.8078.4	PASS	PASS	Succeeded
Firefox	157.0	PASS	PASS	Succeeded
WebKit	27.2	PASS	PASS	Succeeded

The scripts use controlled `fetch` stubs for network responses. They do not establish real refresh-cookie ordering, server-side revocation, behaviour of suspended or background tabs, or interoperability on physical devices, and Playwright's WebKit is an automation build rather than shipped Safari.

4.2.5 Redis persistence, outage and data loss

Disposable Redis 7 instances managed through Docker were used to test four scenarios for both token strategies. Table 4.6 summarizes the recorded behaviour. For the opaque strategy every scenario fails closed. For the PASETO strategy, loss or rollback of Redis state is a documented, unresolved limitation: revoked tokens verify again until they expire. The persistence behaviour itself follows the underlying Redis modes; an RDB snapshot can lose recent writes after a crash, and the append-only file narrows but does not remove that window [19]. Abrupt crash, append-only durability, Sentinel and cluster failover were not tested by the project.

Table 4.6: Redis scenarios and recorded outcomes (project-recorded; Docker-managed tests)

Scenario	Opaque strategy	PASETO strategy
Graceful restart with RDB	Sessions and revocation state preserved	Sessions and deny-list preserved
Temporary outage, fail-closed (default)	All verification rejected (503); recovers when Redis returns	Rejected (503); recovers
Complete data loss (empty restart)	All tokens unknown; users must re-authenticate	Active tokens still verify; previously revoked tokens verify again until expiry
Snapshot rollback	Tokens created after the snapshot fail closed	Tokens revoked after the snapshot are resurrected
Outage with fail-open configured	Refused at configuration time	Valid signatures accepted without deny-list check; audit event emitted

4.2.6 SecureShop demonstration

SecureShop is a demonstration shop application that installs the published @ninshorg/server and @ninshorg/client 0.1.1 packages and runs three authentication paradigms under the same application requirements. Comparison A uses a stateless HS256 JWT issued with the jose library and stored in browser localStorage, with a five-minute expiry and no server-side state. Comparison B uses Ninsho's opaque tokens on two application instances that share a Redis store. Comparison C uses Ninsho with DPoP binding, an in-memory access token, a non-extractable key in IndexedDB, an HTTP-only refresh cookie and cross-tab synchronization. The Security Experiments Lab in the application issues the replay requests so an examiner can see each response. Two accounts are seeded, an administrator and a customer, with demonstration credentials that must never be used in a real system. Figure 4.2 and Figure 4.3 show two of these experiments as captured by the authors.

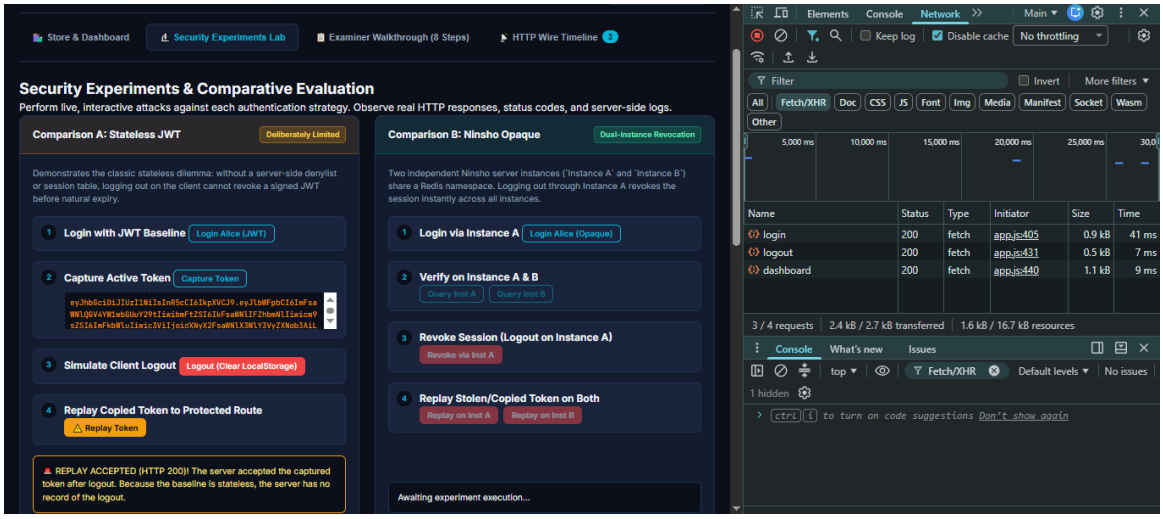


Figure 4.2: SecureShop, Comparison A: replay of a copied JWT after client logout

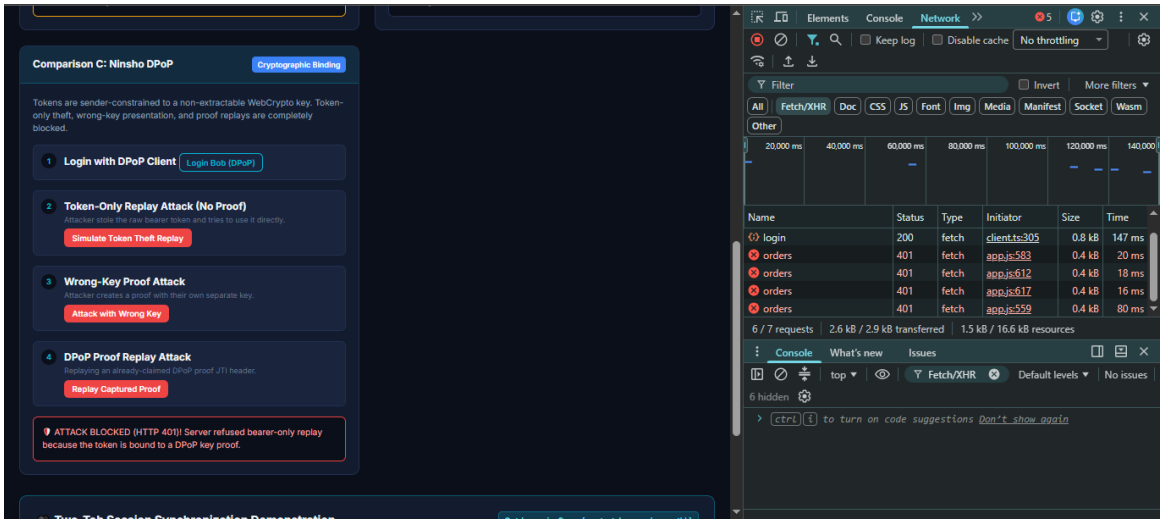


Figure 4.3: SecureShop, Comparison C: token-only, wrong-key and replayed-proof requests rejected

The outcomes documented for SecureShop are given in Table 4.7. The repository records an executed rehearsal for the related three-demo script, in which all ten expected statuses (200, 200, 204, 401, 401, 401, 200, 401, 200, 403) matched on Node 24.19.0 with Redis 7.4.11. SecureShop's own automated tests were re-run on 21 October 2026 (Node 22.13.1) and passed 3 of 3; with one further test added it has 4 tests, all passing.

Table 4.7: SecureShop documented outcomes by comparison

Request	A: stateless JWT baseline	B: Ninsho opaque	C: Ninsho with DPoP
Copied token replayed after client logout	HTTP 200 (accepted until expiry)	HTTP 401 on both instances after server revocation	Server revokes the session
Copied token replayed before logout	Accepted	Accepted (bearer copy works by design)	401 without a proof
Proof signed by a different key	Not applicable	Not applicable	401
Replay of an already used proof	Not applicable	Not applicable	401
Peer tab after logout	No mechanism	No mechanism	Notified through BroadcastChannel

Fresh run, 21 Oct 2026: npx vitest run in demo/viva-showcase — 1 file, 3 tests passed (Node 22.13.1); with one further test added, 4 of 4 passed; log research/validation/logs/secureshop-test.log.

4.2.7 Historical benchmark measurements

The repository contains benchmark reports for release 0.1.0, produced on an Intel Core i5-4300U machine with Node 22.13.1, and an earlier table on a different machine. They are historical measurements of library overhead and not figures for the current commit. Figure

4.4 reproduces two recorded throughput comparisons between Ninsho’s own strategies: in memory, opaque verification was recorded at 17,267 operations per second against 2,922 for PASETO, while with a Redis round trip the figures were 507 and 418. Refresh rotation was recorded at 7,551 operations per second in memory and 43 over Redis loopback. The relative result depends on the hardware and on whether the store is local, which the performance document itself notes.

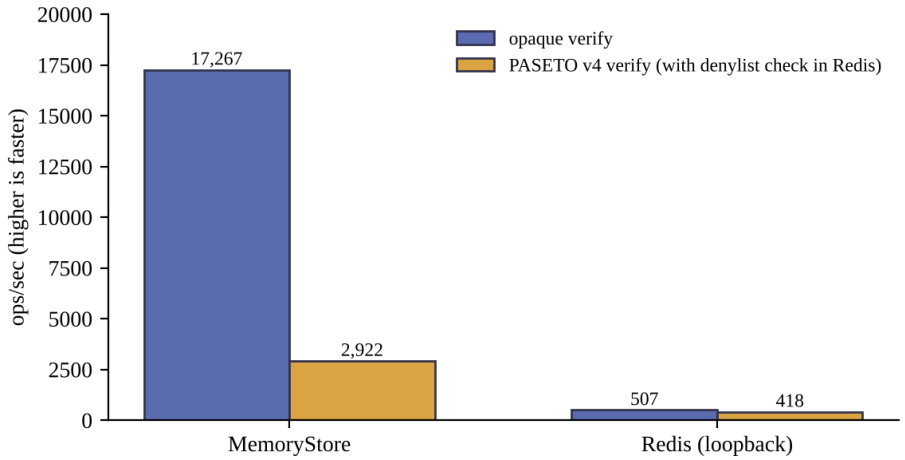


Figure 4.4: Historical verification throughput, release 0.1.0 (not for the current commit)

4.2.8 Organization of the automated test suite

The fresh run of 21 October 2026 executed 54 Vitest test files and one Node.js test file. Table 4.8 shows how they are distributed across the workspaces and what each workspace's tests exercise. Counts are those printed by the test runner for commit 6f82026 before any fix; the column on the right gives the counts after the regression tests added during the final review. Per-file counts were not recorded by the runner and are therefore not reported.

Table 4.8: Organization of the automated tests (fresh run, 21 October 2026)

Workspace	Test files	Tests at 6f82026	After fixes	Main focus
@ninshorg/core	3	97	97	Token generation, hashing, constant-time comparison, error separation, time handling
@ninshorg/server	27	1,116	1,117	Engines, sessions, concurrency, store contract, DPoP, guards, rate limiting, one-time tokens, adapters, Redis persistence
@ninshorg/client	6	117	117	Key storage, lifecycle races, cross-tab synchronization, interoperability with the server
@ninshorg/webauthn	14	623	624	CBOR, COSE, ASN.1, DER, authenticator data, all seven attestation formats, ceremonies, metadata service
examples/express-api	3	101	101	End-to-end HTTP behaviour of the reference integration, passkeys, password reset

Workspace	Test files	Tests at 6f82026	After fixes	Main focus
examples/playground	1	95	95	That each demonstration panel still shows the outcome it claims
manualtest (node:test)	1	1	1	Three HTTP demonstrations against real Redis
Total	55	2,150	2,152	-

The tests fall into five categories. *Unit tests* exercise one module with an in-memory store. *Contract tests* run the same assertions against MemoryStore and RedisStore, so a store implementation is accepted only if it behaves identically, including atomicity. *Concurrency and race tests* start many operations at once or pause one operation at a chosen store call to force a specific interleaving. *Integration tests* drive the Express example over real HTTP. *Environment tests* start disposable Docker Redis containers for persistence scenarios and drive real browsers through Playwright. The categories differ in what they can show: a unit test shows that a branch behaves as intended, while only the contract and race tests against Redis show that a property holds across a network boundary.

4.2.9 Selected test cases

Table 4.9 lists representative test cases for the main security properties. The test names are quoted from the repository; the recorded result is from the fresh run of 21 October 2026 (all passed) unless stated otherwise.

Table 4.9: Selected test cases with setup, expected result and recorded outcome

ID	Test (file)	Setup	Expected result	Recorded
TC-01	never stores the raw token (opaque-engine)	Issue a token; inspect every stored value	Raw token appears nowhere in the store	Pass
TC-02	ends the session for both parties when a stolen token is redeemed first (session)	Attacker rotates first; victim presents the old token	Family revoked; both parties signed out	Pass
TC-03	does not raise a reuse alarm for a token rotated moments ago (session)	Second tab inside the grace window	Same replacement returned; no alarm	Pass
TC-04	lets exactly one of many simultaneous clicks win (one-time-token)	Many concurrent consumptions of one reset token	Exactly one succeeds	Pass
TC-05	refuses a stolen token presented with no proof at all (dpop-integration)	Bound token without a DPoP header	401	Pass
TC-06	refuses a captured proof replayed with the token it came from (dpop-integration)	Same proof sent twice	Second request 401	Pass
TC-07	refuses a caller addressing someone else's resource (middleware)	requireOwner with a different user id	403	Pass

ID	Test (file)	Setup	Expected result	Recorded
TC-08	rejects rather than admitting a request it could not check (store-invariants)	Limiter pointed at an unreachable Redis	StoreUnavailableError	Pass*
TC-09	preserves active opaque sessions and their revocation state across a graceful restart (redis-persistence)	Docker Redis with RDB; SHUTDOWN SAVE; restart	Sessions and revocations survive	Pass
TC-10	PASETO: previously revoked tokens become usable again (redis-persistence)	Revoke, then restart Redis without data	Revoked token verifies again (documented limitation)	Pass

*TC-08 failed in the first local attempt because the assessor had started Redis on port 6390, which the test uses as its deliberately unreachable address; with Redis on port 6379 it passed. The incident is recorded because it shows how a local environment can produce a misleading failure.

4.2.10 Regression tests added during the final review

The review described in Section 4.5 added regression tests for three defects, each written first and observed to fail on the unmodified code. For F-01 (PASETO session index), a test in paseto-engine.test.ts showed a revoked session's token being accepted, and a probe through the public API against real Redis reproduced it; after the fix both refused the token. For B-01 (WebAuthn owner), two tests in the WebAuthn server suite showed an attacker signing in as another user; after the fix the server refuses a credential without a stored owner. For B-12 (SecureShop logout), a demonstration test showed the access token still accepted after logout; after the fix all four demonstration tests passed. These fixes exist in the working tree of commit 6f82026 and have not been committed or published: the npm packages at version 0.1.1 were built from commit f784566, whose package sources differ from 6f82026 only in version numbers, so the published packages still contain all three defects.

4.2.11 Fresh benchmark: objectives, configurations and procedure

The benchmark answers two separate questions. The first is the *verification cost* of one access token in-process, and how much of it is the store round trip. The second is the *throughput and latency of protected HTTP requests* when different configurations offer the same server-side revocation guarantee. The harness is in research/validation/benchmarks and its method is documented in METHODOLOGY.md in the same folder. Table 4.10 lists the configurations; only the first three are compared with each other.

Table 4.10: Benchmark configurations and the checks each performs per request

Configuration	Role	Per-request checks
ninsho-opaque	Compared	SHA-256 of the token, one Redis GET (presence means not revoked), record checks; 503 on store error
ninsho-paseto	Compared	Ed25519 PASETO v4.public signature, claim and time checks, one Redis EXISTS on the deny-list; 503 on store error
jwt-redis	Compared	jose library EdDSA JWT verification with issuer, audience and time checks, one Redis EXISTS on a deny-list; 503 on Redis error
jwt-stateless	Reference only	As jwt-redis without the deny-list: a revoked token is accepted until it expires
no-auth	Reference only	No authentication; the floor for Express, loopback HTTP and JSON on this machine

The signed configurations use the same algorithm, Ed25519, so that the comparison is not between different signature schemes by accident. Every compared configuration performs exactly one Redis round trip per request and refuses requests when Redis is unavailable. Before any timing, in every trial, each configuration was sent a valid token, no token, a tampered token, a revoked token and, for signed formats, a token for another audience; a compared configuration that failed to refuse any of these would have stopped the run. All fifteen preflight checks passed, and the stateless reference accepted the revoked token, as designed. DPoP was excluded because a fair comparison would place a signing operation in the load generator on the same two processor cores, and the JWT baseline has no equivalent.

The reported run used the harness's standard mode: three trials; for verification, 200 warm-up and 1,000 measured operations per configuration per trial; for HTTP, a separate server process per configuration, 2 seconds of warm-up and 5 seconds of measurement at 1 and 16 concurrent clients, each client keeping one request in flight over a keep-alive connection. Configurations ran in a shuffled order recorded in the run log. The environment is given in Table 4.2; the working tree contained only the F-01 change, which alters a store lifetime at token issuance and no verification path.

4.2.12 Fresh benchmark: verification cost

Table 4.11 reports, for each operation, the median across trials of each trial's median (p50) latency, the range of the trial medians and the median of the trials' 95th percentiles. Figure 4.5 shows each trial separately.

Table 4.11: Verification cost at one operation at a time (milliseconds)

Operation	p50 median	p50 range	p95 median
Redis EXISTS (ioredis)	1.337	1.088–1.459	3.688
Redis GET (Ninsho store)	1.357	1.245–1.538	8.083
Opaque + Redis	1.588	1.200–1.743	7.679
PASETO + deny-list	2.051	2.043–2.217	5.938
JWT + deny-list	2.163	2.094–2.350	3.934
PASETO, signature only (reference)	0.323	0.241–0.491	0.801
JWT, signature only (reference)	0.320	0.301–0.513	0.603

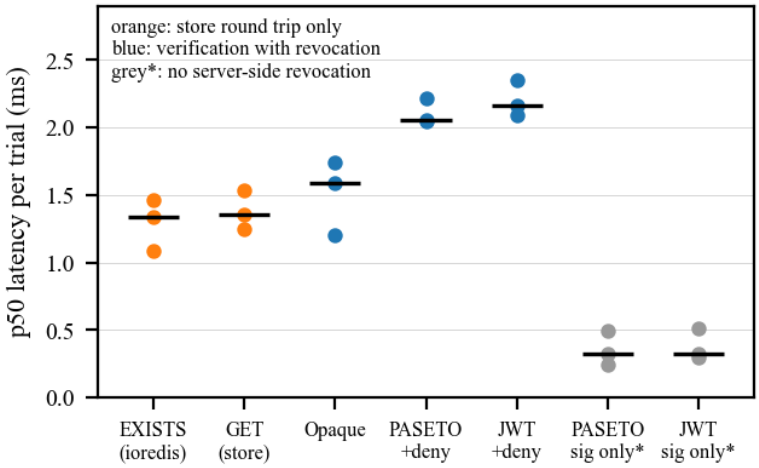


Figure 4.5: Per-trial median verification latency (dots) and the median of the three trials (bars)

In this environment every verifier that checks revocation spent most of its time on the store round trip. The opaque verifier exceeded a bare Redis GET by about 0.2 ms, and the two signed verifiers exceeded a bare EXISTS by about 0.7 to 0.8 ms, of which the signature check alone accounts for roughly 0.3 ms. The latency distributions have long tails: some trials recorded 99th-percentile store latencies above 15 ms, which reflects the network path from Windows into Docker's WSL2 virtual machine rather than the library.

4.2.13 Fresh benchmark: HTTP throughput and variability

Table 4.12 gives the throughput of successful responses; no request failed during measurement. At one client the ranges of the three compared configurations overlap (opaque 165–302, PASETO 226–275, JWT with Redis 213–261 requests per second), so no ordering is established. At 16 clients the opaque configuration reached a median of 951 requests per second with a narrow range (933–978), above JWT with Redis (801; 577–804) and PASETO (564; 420–567). Figure 4.6 shows the three trials of each cell. The stateless

JWT reference was roughly twice as fast as the compared configurations at one client; this is the cost of the revocation lookup it omits, not an equivalent alternative.

Table 4.12: Protected-request throughput (successful requests per second)

Configuration	1 client median	1 client CV	16 clients median	16 clients CV
Ninsho opaque	287	29.8%	951	2.4%
Ninsho PASETO	245	9.9%	564	16.2%
JWT + Redis deny-list	251	10.4%	801	17.9%
JWT stateless (reference)	488	10.0%	963	34.1%
No authentication (reference)	1,136	5.0%	1,310	5.4%

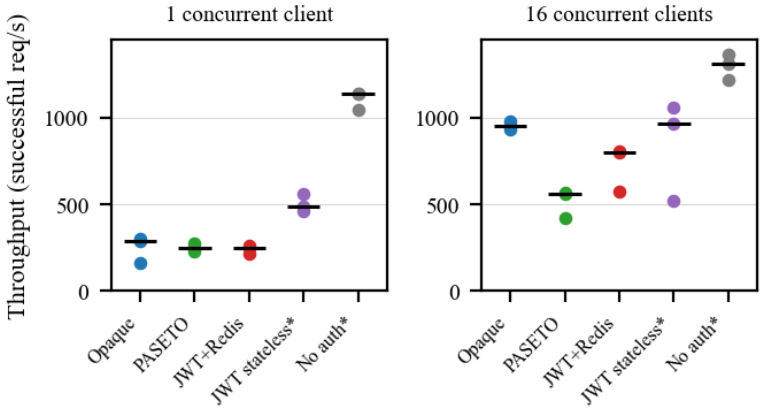


Figure 4.6: Per-trial throughput (dots, 5 s each) and medians (bars); starred configurations are references without an equivalent revocation guarantee

Variability was substantial: the coefficient of variation (CV) of throughput reached 29.8 % among the compared configurations and 34.1 % for the stateless reference. Figure 4.7 explains part of it. The seeded shuffle happened to run PASETO fourth and JWT with Redis fifth in all three trials, so these two configurations always ran last, and the third trial, in which both dropped, is consistent with the machine slowing over the run. Their difference at 16 clients is therefore confounded with run position, and no profiling was done to explain it. The results describe one laptop, one JWT library and a Redis reached across a virtualization boundary; they do not show that opaque tokens or Ninsho are faster than JWT-based systems in general.

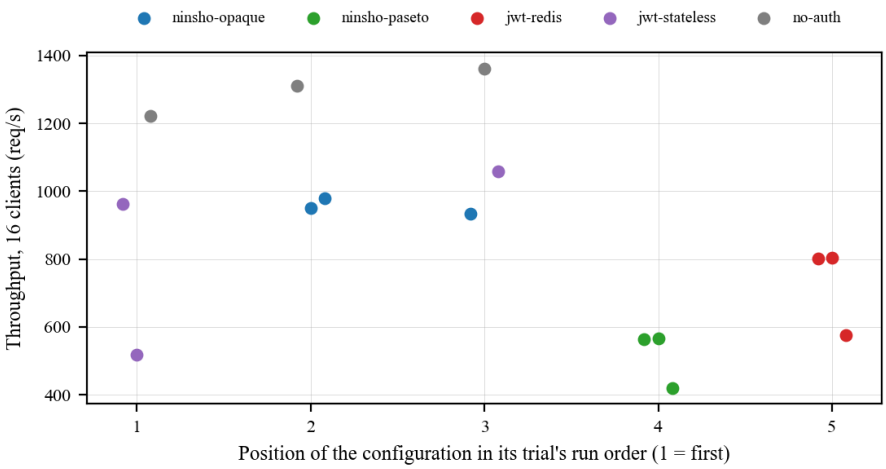


Figure 4.7: Throughput at 16 clients against the configuration's position in its trial's run order

4.3 Comparison with Existing Methods

Table 4.13 compares the design of three approaches. The first column is the deliberately limited JWT baseline used in SecureShop, which is a teaching device. A JWT-based system can add a deny-list, a per-user session version, introspection or sender-constrained tokens, and the standards describe how [3][5][9]. The comparison therefore shows what each baseline provides without extra infrastructure; it does not rank JWT and Ninsho in general.

Table 4.13: Comparison of the SecureShop baseline with Ninsho configurations (design properties and documented outcomes)

Property	A: stateless JWT baseline	B: Ninsho opaque	C: Ninsho with DPoP
Server state per token	None	One record per token in the store	One record per token, plus proof identifiers
Revocation before expiry	Not available without added state	Immediate across instances sharing the store	Immediate; peer tabs notified
Use of a copied token elsewhere	Works until expiry	Works until revoked	Rejected without the bound key
Replay of a captured proof	Not applicable	Not applicable	Rejected (identifier claimed once)
Behaviour if the store is lost	Not applicable	Fails closed	Fails closed (opaque); PASETO variant can resurrect revoked tokens
Per-request work	Signature check	One store read	Store read, proof check and an atomic claim; client signs
Protects against malicious script in the page	No	No	No; script in the origin can request signatures

The per-request work row describes design and is not a measurement. The last row is a limitation that the project states explicitly: DPoP with a non-extractable key prevents key export, not use of the key by hostile code running in the same origin.

4.4 Limitations and Threats to Validity

- **Test counts are configuration-specific. Earlier documents record 2,116 and 2,142 for other working-tree states; the fresh count for this commit is 2,150; with the regression tests added during the final review it is 2,152, plus 4 SecureShop tests run separately.**
- **Single local re-run. The fresh results come from one Windows machine on Node 22; Node 20 and Linux were covered only by the CI run on the examined commit.**
- **Version drift.** The manual attack procedures and benchmarks were recorded against release 0.1.0.
- **Stubbed browser networking, automation browsers and no physical devices.** See Section 4.2.
- **CI coverage of the Docker persistence suite is not established** for the reason given in Section 4.2.1.
- **Demonstration baseline.** The JWT baseline is deliberately limited, and the demonstration accounts and in-memory data are for illustration.
- **Internal validity of the manual tests.** Some attack scripts exercise the example applications and not every deployment configuration.
- Benchmark scope. Three trials of 5 seconds per configuration on one laptop, with seven unrelated containers running and the load generator, server and Redis sharing two processor cores; configuration order was not balanced.
- Security review. The review in Section 4.5 was performed with the assistance of an AI coding assistant working with the project's own tests. It is an internal review, not an independent third-party audit, and its coverage of the WebAuthn attestation formats and parsers relied mainly on their existing test suites.
- Unreleased fixes. The fixes for F-01, B-01 and B-12 are in the working tree only; the published packages at version 0.1.1 do not contain them, and continuous integration has not run on them.

4.5 Security Evaluation Against the Threat Model

This section relates the threat model of Section 3.6 to the evidence. Four kinds of evidence are distinguished: *security reasoning* about the design, *automated regression tests* that exercise a property, *source review* in which code was read and suspected issues were reproduced with probe scripts against the built packages, and an *independent audit* by a qualified third party, which has not been carried out. Table 4.14 maps each security objective to its control, evidence and residual risk.

Table 4.14: Security objectives, controls, evidence and residual risk

Objective	Control	Evidence	Residual risk
O1 Revoked credential refused everywhere	Shared store; record deletion or deny-list; marker re-checks	Revocation and race tests against Redis; SecureShop B	PASETO revocations lost if Redis data is lost; F-01 in 0.1.1
O2 Refresh replay ends the session	Rotation, tombstones, family revocation	Session and concurrency tests (TC-02, TC-03)	Unbound token replayed inside the 30 s grace window is not detected; F-03 false alarm
O3 Bound token needs a fresh proof	DPoP proof checks and replay guard	dpop-integration and dpop-request tests; SecureShop C	No server nonce; same-origin script can sign; F-04
O4 Refuse when state is unavailable	Fail-closed default; configuration refusals	store-invariants and persistence tests	Operator may choose fail-open for PASETO
O5 Guards refuse when unsure	Typed selectors; deny on missing data	middleware and fresh-auth tests	Guards not mounted by the application; F-02 for rate limits
O6 Store holds no usable credential	SHA-256 hashing	never stores the raw token (TC-01)	Grace mapping stores one raw token for 30 s

4.5.1 Findings of the review

The review read the session manager, both token engines, the Redis store, the HTTP and DPoP middleware, the rate limiter, single-use tokens, the browser client and the WebAuthn ceremony code in full, and other modules in part. Every suspected issue was reproduced with a probe script against the built packages before it was classified as a finding; suspicions that were refuted, such as a backup-flag check that the authenticator-data parser does enforce, are not listed. A second review pass, run separately, proposed further hypotheses, two of which (B-01 and B-12) were then reproduced and fixed. Table 4.15 lists all confirmed findings and their status.

Table 4.15: Confirmed findings and their status

ID	Finding	Severity	Status
F-01	PASETO: a revoked session's token keeps verifying when the clock tolerance exceeds 60 s, because the session index expires first	High impact, low likelihood (non-default setting)	Fixed in working tree with regression test; present in 0.1.1

ID	Finding	Severity	Status
F-02	trustProxy: false still honours X-Forwarded-For when the framework itself trusts proxies, through req.ip	Medium	Open
F-03	A logout racing a rotation can leave a tombstone, so a stale tab raises a false reuse alarm	Low to medium	Open
F-04	One DPoP key can yield several thumbprints because key coordinates are not decoded canonically	Low	Open
F-05	The CI check for the persistence suite cannot tell a run from a skip	Low (evidence)	Open
F-06	The Express example inherits a critical proxy-addr advisory through Express 5.2.1	Informational	Open; not in published packages
F-07	The client refuses same-origin paths containing //	Low (fails safe)	Open
F-08	Persistence tests can fail when Docker reuses a container name	Low (test reliability)	Open
B-01	WebAuthn sign-in could return another account when the stored credential's owner was omitted, because the user handle is not signed	Medium to high (integration-dependent)	Fixed in working tree with regression tests; present in 0.1.1
B-12	SecureShop's DPoP logout passed the refresh cookie as a session id, so the session was not revoked	Medium (demonstration only)	Fixed in working tree with regression test

The findings have a common pattern. Most lie at a boundary between Ninsho and something it does not own: a web framework's proxy setting (F-02), the encoding of a client-supplied key (F-04), the continuous-integration runner (F-05), a container runtime (F-08), or the way an application stores passkey owners (B-01). F-01 is instructive in another way: an earlier fix had already aligned the PASETO deny-list lifetime with the clock tolerance, and the sibling index was missed. A suite of more than two thousand passing tests did not prevent any of these, which is why this report treats the tests as scoped evidence rather than proof of security.

4.5.2 Implications for deployment

Redis state loss and rollback. With the opaque strategy, losing Redis data ends every session, which is disruptive but safe. With the PASETO strategy, losing the deny-list or restoring an older snapshot makes revoked tokens valid again until they expire, as TC-10 demonstrates; the five-minute default access lifetime bounds the window. Deployments that choose PASETO should therefore run Redis with persistence appropriate to their recovery objectives [19].

Fail-open. Selecting `onStoreError: 'open'` with PASETO admits tokens on their signature alone during an outage. It also disables DPoP replay protection and rate limiting for the same period. Each admission emits a `store.unavailable` event so that the period

can be reconstructed, but the security consequence is real and the default remains fail-closed.

Concurrency. Session creation, rotation and revocation are sequences of store operations, not transactions. The marker-first ordering and the re-checks close the interleavings that were found and tested; F-03 shows that at least one benign interleaving remains, and logout-all intentionally works on one snapshot, so a login completed after the snapshot survives it.

Proxy trust and replay. Per-address rate limiting and DPoP URI binding both depend on what the application believes about its proxies. F-02 shows that the two layers of configuration, the framework's and Ninsho's, can disagree. The replay guard bounds a captured proof to one use within its 60-second window; it does not provide the server-chosen freshness of a DPoP nonce, which is not implemented.

4.6 Critical Discussion

The evidence supports a specific set of statements. Under the tested conditions, the default configuration revokes sessions immediately across instances that share Redis, rotates refresh tokens with reuse detection without signing out concurrent tabs, refuses rather than guesses when Redis is unavailable, and refuses token-only, wrong-key and replayed-proof requests when DPoP is enabled. The behaviour of each strategy under Redis restart, outage, data loss and rollback is measured, including the unfavourable PASETO case. On the measured machine, the store round trip, not the token format, accounted for most of the verification cost among configurations with equal revocation guarantees.

The evidence does not support several stronger statements that a reader might be tempted to draw, and Table 4.16 lists what would be needed to support them.

Table 4.16: Open questions and the evidence that would be needed

Question	Current evidence	Evidence needed
Is Ninsho faster than JWT-based systems?	Three 5-second trials on one laptop; order confound; one JWT library	Linux hosts, local Redis, balanced order, more and longer trials, profiling, several libraries
Is Ninsho secure?	Tests, an AI-assisted internal review, reproduced findings	Independent third-party security review; resolution of open findings
Does it work on Node.js 20 and Linux after the fixes?	CI on 6f82026 before the fixes; local Node 22 on Windows after	Commit the fixes and run continuous integration
Do the browser guarantees hold on real devices?	Automation builds with stubbed networking	Physical devices, real cookie ordering, suspended tabs
Is it usable by other developers?	Example applications written by the authors	A study with developers unfamiliar with the library

Question	Current evidence	Evidence needed
Does PASETO revocation survive crashes?	Graceful restart and data-loss scenarios	Abrupt crash, append-only persistence, replica failover tests

Chapter 5

Conclusion and Future Scope

5.1 Conclusion

This project designed and evaluated Ninsho, an authentication and session-management SDK for Node.js. The implemented system provides opaque access tokens with single-read verification and immediate revocation, single-use refresh tokens with a grace window and reuse detection, optional PASETO v4.public tokens, optional DPoP proof-of-possession with atomic replay protection, authorization guards, rate limiting, framework adapters, a browser client with a non-extractable key and cross-tab invalidation, and an optional WebAuthn verifier, all over an injected store with a Redis implementation.

The evidence examined supports these behaviours under the tested conditions: the project's own runs and the GitHub Actions run on the examined commit show a passing suite with real Redis and passing Chromium, Firefox and WebKit regressions; recorded manual attack procedures were rejected as expected; and the Redis scenarios document how each strategy fails. Table 5.1 relates the objectives of Section 1.4 to the evidence.

Table 5.1: Objectives and the evidence supporting them

Objective	Status	Evidence and caveat
1. Study standards and research	Met	Chapter 2; sources verified against publisher pages
2. Design and implement the SDK	Met	Repository at commit 6f82026; packages published as 0.1.1
3. Browser client with non-extractable key and cross-tab invalidation	Met, with limits	Browser regressions on three engines; stubbed networking; best-effort delivery
4. Evaluate by tests, attacks, browsers, Redis and demonstration	Met; re-executed locally	Fresh local re-run (21 Oct 2026) of tests, browsers, Redis scenarios and SecureShop; regression tests added for the defects it found
5. Compare a limited JWT baseline with Ninsho	Met, narrowly	SecureShop outcomes, plus a single-machine benchmark against a JWT baseline with equivalent Redis revocation (Sections 4.2.11 to 4.2.13)
6. Document limitations	Met	Section 4.4 and the list below

The results are scoped. They do not show that Ninsho is more secure or faster than JWT-based systems in general. The following limitations should be read with them:

- Ninsho is pre-1.0 and is not certified.

- With the PASETO strategy, loss or rollback of Redis state revives revoked tokens until expiry; with fail-open configuration, revocation and replay checks are skipped during an outage. Opaque mode fails closed.
- Cross-tab notices are best-effort and unauthenticated by design; a failed delivery leaves a peer tab to discover the logout from the server.
- DPoP with a non-extractable key does not defend against malicious script in the same origin.
- Session creation, rotation and revocation are multi-step operations, not one transaction: a failed rotation can consume the old refresh token, and logout-all works on a snapshot of indexed sessions.
- The immediate-login registration route of the example reveals account existence through its response; applications that need enumeration resistance should verify email first.
- Physical-device passkey interoperability, abrupt Redis crash, Sentinel and cluster behaviour, suspended-tab behaviour and real-cookie ordering were not tested.
- There is no DPoP server-nonce mechanism in the implementation reviewed; DPoP replay protection relies on the proof identifier and acceptance window.

5.2 Technical Contributions

- A session-management library whose shortest configuration is its safest: opaque tokens, fail-closed store handling and five-minute access tokens, with unsafe combinations refused at start-up.
- A refresh-rotation design whose store-operation ordering keeps concurrent tabs signed in while still detecting replay, together with a revocation marker that closes the races between logout and issuance, each pinned by regression tests against real Redis.
- DPoP sender-constraining in which the refresh token's binding is checked before it is consumed, so that a thief without the key cannot end the victim's session.
- Measured documentation of how each token strategy behaves under Redis restart, outage, data loss and rollback, including the unfavourable PASETO case.

- A browser client that keeps credentials out of script-readable storage and coordinates logout across tabs without sharing credentials.
- An evidence practice in which security claims name their tests, evidence is labelled by source and commit, and a reproducible benchmark refuses to time any configuration that fails a security preflight.

5.3 Lessons Learned

The most consistent lesson of the project is that passing tests describe what was tested, not what is true. Several defects were found by exercising a code path directly, outside the happy path the existing tests followed, and each looked correct on reading. Ordering of store operations turned out to matter more than any cryptographic choice: the behaviour of refresh rotation, revocation and grace adoption depends on which write becomes visible first. Local results also proved unreliable as evidence on their own; a stale build, a single Node.js version, an absent Redis or, in this project's final run, a port collision each produced results that would have been misleading if reported without checking. Finally, honest comparison requires choosing baselines that provide the same guarantee: a stateless JWT is faster because it does less, and reporting it beside a revocable session without that qualification would misrepresent both.

5.4 Future Scope

- Re-run all manual attack procedures against the final release.
- Re-run the full suite, browser matrix and benchmarks on a documented machine, record logs with the commit hash, and publish a reproducibility package.
- Repeat the single-machine performance comparison of Sections 4.2.11 to 4.2.13 on Linux hosts and dedicated hardware, with more trials and confidence intervals.
- Add durability options, such as guidance and tests for append-only persistence, Sentinel and cluster failover, and consider a signed or replicated deny-list for the PASETO strategy.
- Evaluate usability with developers unfamiliar with the library, measuring time to integrate and configuration errors.
- Extend the passkey work with a physical-authenticator interoperability matrix.

- Investigate a server-nonce option for DPoP and an authenticated channel for cross-tab notices.

Table 5.2 orders the future work by priority, with the reason for each priority.

Table 5.2: Prioritized future work

Priority	Work item	Reason
1	Commit the fixes for F-01, B-01 and B-12, run continuous integration and publish a patch release	The published 0.1.1 packages contain these defects
2	Fix F-02 (proxy trust through req.ip) and F-03 (stale tombstone)	Open findings with security or alarm-integrity impact
3	Commission an independent security review	The current review is internal and model-assisted
4	Repeat the benchmark on Linux with balanced order, more trials and profiling	Current results are exploratory and confounded
5	Add durability tests for append-only persistence and replica failover	PASETO revocation depends on store durability
6	Offer IPv6 prefix aggregation and DPoP server nonces	Strengthen rate limiting and proof freshness
7	Study integration effort with developers new to the library	No usability evidence exists
8	Build a physical-authenticator interoperability matrix	Passkey support is tested only with virtual authenticators

5.5 Validation Required Before Production-Readiness Claims

Ninsho is pre-1.0 and should not be described as production-ready on the present evidence. At minimum, the following would be needed before such a claim could be made:

- the fixes released and verified by continuous integration on every supported Node.js version;
- an independent security review with its findings resolved or documented;
- operational testing with the persistence and failover configuration of the intended deployment;
- performance measurement on representative hardware with the intended Redis topology; and
- a documented process for receiving and responding to vulnerability reports.

Chapter 6

List of Publications

6.1 List of Published Papers

None at the time of writing. A conference-format research manuscript describing Ninsho has been drafted in the project repository (research/paper); it has not been submitted to any venue and has not been peer-reviewed. [Students to confirm; if it is submitted or posted as a preprint, add its status, venue or repository, year and DOI.] The npm releases of the software packages (0.1.0 and 0.1.1) are software releases and are not counted as publications.

6.2 Details of Participation in Project Competitions

None at the time of writing. [Students to confirm; add the competition name, organizer, date and outcome if applicable.]

References

[1] Ninsho project, source repository, commit 6f82026 (release v0.1.1), 2026. [Online]. Available: https://github.com/NinshoORG/ninsho

[2] M. Jones and D. Hardt, “The OAuth 2.0 Authorization Framework: Bearer Token Usage,” RFC 6750, IETF, Oct. 2012, doi: 10.17487/RFC6750.

[3] M. Jones, J. Bradley, and N. Sakimura, “JSON Web Token (JWT),” RFC 7519, IETF, May 2015, doi: 10.17487/RFC7519.

[4] T. Lodderstedt, J. Bradley, A. Labunets, and D. Fett, “Best Current Practice for OAuth 2.0 Security,” RFC 9700 (BCP 240), IETF, Jan. 2025, doi: 10.17487/RFC9700.

[5] D. Fett, B. Campbell, J. Bradley, T. Lodderstedt, M. Jones, and D. Waite, “OAuth 2.0 Demonstrating Proof of Possession (DPoP),” RFC 9449, IETF, Sep. 2023, doi: 10.17487/RFC9449.

[6] W3C, “Web Authentication: An API for accessing Public Key Credentials – Level 3,” W3C Recommendation, Aug. 25, 2026. [Online]. Available: https://www.w3.org/TR/webauthn-3/

[7] OWASP Foundation, “OWASP Top 10 API Security Risks – 2023.” [Online]. Available: https://api-security.owasp.org/editions/2023/en/0x11-t10/

[8] J. Yang, E. Wang, J. Chen, Q. Wang, Y. Zhang, H. Duan, W. Xie, and B. Wang, “Token time bomb: Evaluating JWT implementations for vulnerability discovery,” in Proc. Network and Distributed System Security Symp. (NDSS), 2026. [Online]. Available: https://www.ndss-symposium.org/ndss-paper/token-time-bomb-evaluating-jwt-implementations-for-vulnerability-discovery/

[9] Y. Sheffer, D. Hardt, and M. Jones, “JSON Web Token Best Current Practices,” RFC 8725 (BCP 225), IETF, Feb. 2020, doi: 10.17487/RFC8725.

[10] J. Bonneau, C. Herley, P. C. van Oorschot, and F. Stajano, “The quest to replace passwords: A framework for comparative evaluation of web authentication schemes,” in Proc. IEEE Symp. Security and Privacy (S&P), 2012.

[11] M. Squarcina, P. Adão, L. Veronese, and M. Maffei, “Cookie crumbles: Breaking and fixing web session integrity,” in Proc. 32nd USENIX Security Symp., Anaheim, CA, USA, Aug. 2023, pp. 5539–5556.

[12] D. Fett, R. Küsters, and G. Schmitz, “A comprehensive formal security analysis of OAuth 2.0,” in Proc. ACM SIGSAC Conf. Computer and Communications Security (CCS), 2016; arXiv:1601.01229.

[13] D. Hardt, Ed., “The OAuth 2.0 Authorization Framework,” RFC 6749, IETF, Oct. 2012, doi: 10.17487/RFC6749.

[14] PASETO Standard, “PASETO: Platform-Agnostic SEcurity TOkens – specification,” GitHub repository. [Online]. Available: https://github.com/paseto-standard/paseto-spec

[15] M. Jones and N. Sakimura, “JSON Web Key (JWK) Thumbprint,” RFC 7638, IETF, Sep. 2015, doi: 10.17487/RFC7638.

[16] C. Bormann and P. Hoffman, “Concise Binary Object Representation (CBOR),” RFC 8949 (STD 94), IETF, Dec. 2020, doi: 10.17487/RFC8949.

[17] L. Lassak, E. Pan, B. Ur, and M. Golla, “Why aren’t we using passkeys? Obstacles companies face deploying FIDO2 passwordless authentication,” in Proc. 33rd USENIX Security Symp., 2024, pp. 7231–7248.

[18] S. Josefsson and I. Liusvaara, “Edwards-Curve Digital Signature Algorithm (EdDSA),” RFC 8032 (Informational), IETF, Jan. 2017, doi: 10.17487/RFC8032.

[19] Redis Ltd., “Redis persistence,” Redis documentation. [Online]. Available: https://redis.io/docs/latest/operate/oss_and_stack/management/persistence/

[20] T. Lodderstedt, S. Dronia, and M. Scurtescu, “OAuth 2.0 Token Revocation,” RFC 7009, IETF, Aug. 2013, doi: 10.17487/RFC7009.

[21] J. Richer, “OAuth 2.0 Token Introspection,” RFC 7662, IETF, Oct. 2015, doi: 10.17487/RFC7662.

[22] S. Calzavara, R. Focardi, M. Squarcina, and M. Tempesta, “Surviving the web: A journey into web session security,” ACM Computing Surveys, vol. 50, no. 1, art. 13, pp. 1–34, 2017, doi: 10.1145/3038923.

[23] S. Sivakorn, I. Polakis, and A. D. Keromytis, “The cracked cookie jar: HTTP cookie hijacking and the exposure of private information,” in Proc. IEEE Symp. Security and Privacy (S&P), 2016, doi: 10.1109/SP.2016.49.

[24] F. Skokan, “jose: JWA, JWS, JWE, JWT, JWK, JWKS for Node.js and other runtimes,” software library, version 5.10.0. [Online]. Available: https://github.com/panva/jose